



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТвАЭ)

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ
Ознакомительная практика

приказ Университета о направлении на практику от «10» февраля 2026 г. №1028-С

Отчет представлен к
рассмотрению:

Студент группы ИКБО-50-25

«25» мая 2026 г.

А.В. Базинен
(подпись и расшифровка подписи)

Отчет утвержден.
Допущен к защите:

Руководитель практики
от кафедры

«25» мая 2026 г.

В.М. Рогоза
(подпись и расшифровка подписи)

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТвАЭ)
ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ
Ознакомительная практика

Студенту 1 курса учебной группы ИКБО-50-25

Багизян Артур Варданович

Место и время практики: РТУ МИРЭА кафедра ИТвАЭ, с 09 февраля 2026 г. по 30 мая 2026 г.

Должность на практике: студент

1. СОДЕРЖАНИЕ ПРАКТИКИ:

- 1.1. Изучить: изучить материал в соответствии с выданной темой
1.2. Практически выполнить: разработать информационный контент в виде тематического реферата в двух частях в формате .docx объемом не менее 30 страниц (без учета титульного листа, содержания, списка источников) на основе материала, выбранного в открытых информационных ресурсах сети Интернет; подготовить презентацию в формате .pptx (10 слайдов); краткий доклад по презентации 5-8 листов.

ТЕМА: «Современные мобильные операционные системы, современные приложения и перспективы развития приложений для мобильных устройств разработки ИТ предприятий Отрасли»

- 1.3. Ознакомиться: со специализированным открытым программным обеспечением

2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: нет

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в соответствии с методическими указаниями по учебной практике по направлению 09.03.04 «Программная инженерия»

Руководитель практики от кафедры
«09» февраля 2026 г.

В.Рогоза
Подпись

(Рогоза В.М.)

Задание получил
«09» февраля 2026 г.

А.В.Багизян
Подпись

(Багизян А.В.)

СОГЛАСОВАНО:

Заведующий кафедрой:

«09» февраля 2026 г.



Подпись

(Боридько С.И.)

Проведенные инструкта:

Охрана труда:

Инструктирующий



Подпись

«09» февраля 2026 г.

Боридько С.И.,
зав.каф. ИТвАЭ

Инструктируемый



Подпись

Базинен А.В.

Техника безопасности:

Инструктирующий



Подпись

«09» февраля 2026 г.

Боридько С.И.,
зав.каф. ИТвАЭ

Инструктируемый

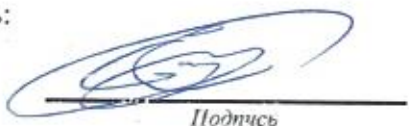


Подпись

Базинен А.В.

Пожарная безопасность:

Инструктирующий



Подпись

«09» февраля 2026 г.

Боридько С.И.,
зав.каф. ИТвАЭ

Инструктируемый



Подпись

Базинен А.В.

С правилами внутреннего распорядка ознакомлен:



Подпись

«09» февраля 2026 г.

Базинен А.В.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**РАБОЧИЙ ГРАФИК
ПРОВЕДЕНИЯ ОЗНАКОМИТЕЛЬНОЙ ПРАКТИКИ**

студента Базилевича А.В. 1 курса группы ИКБО-50-25 очной формы обучения,
обучающегося по направлению подготовки 09.03.04 Программная инженерия профиль
«Информационные технологии в атомной отрасли»

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	09.02.2026	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения производственной практики, инструктаж по технике безопасности, получение задания на практику)	В.Роща
1-3	09.02.2026-28.02.2026	Исследовательский этап (формирование 1-й части реферата)	В.Роща
4	02.03.2026	Представление руководителю структурированного материала: 1-й части реферата	В.Роща
4-6	02.03.2026-21.03.2026	Технологический этап (Формирование 2-й части реферата и разработка шаблона презентации)	В.Роща
7	23.03.2026	Представление руководителю (Формирование 2-й части реферата и разработка шаблона презентации)	В.Роща
7-9	23.03.2026-11.04.2026	Исследовательский раздел (Формирование презентации)	В.Роща
10	13.04.2026	Представление руководителю структурированного материала: презентации по теме	В.Роща
10-12	13.04.2026-02.05.2026	Технологический этап (доклад по презентации)	В.Роща
13	04.05.2026	Представление руководителю доклада по теме	В.Роща
16	30.05.2026	Подготовка окончательной версии отчета и презентации с докладом по теме практики (Оформление материалов отчета в полном соответствии с требованиями на оформление письменных учебных работ студентов, загрузка отчета)	В.Роща

Руководитель практики от
кафедры

В. Рогоза /Рогоза В.М./

Обучающийся

Башиев А.В. /Башиев А.В. /

Согласовано:

Заведующий кафедрой

Боридько С.И. /Боридько С.И., д.т.н, профессор/



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра Информационных Технологий в Атомной Энергетике (ИТАЭ)

Реферат по теме

«Современные мобильные операционные системы, современные приложения и перспективы развития приложений для мобильных устройств разработки ИТ предприятий в Атомной Отрасли»

Выполнил студент группы ИКБО-50-25

Багинян А.В..

Москва 2026

СОДЕРЖАНИЕ

Введение	10
1. Обзор современных мобильных ОС	12
1.1. Android	12
1.1.1. История и архитектура	12
1.1.2. Экосистема и доля рынка	13
1.1.3. Версии, фрагментация и модель обновлений	14
1.1.4. Безопасность и управление устройствами (MDM/EMM)	15
1.2. iOS	16
1.2.1. История и архитектура	16
1.2.2. Закрытая экосистема Apple: плюсы и минусы	18
1.2.3. Политика безопасности и конфиденциальности	19
1.2.4. Корпоративный профиль и Apple Business Manager	19
1.3. Российские мобильные ОС на базе AOSP	20
1.3.1. Предпосылки создания отечественных ОС	20
1.3.2. KVADRA_OS	21
1.3.3. РЕД ОС М	22
1.3.4. Общие проблемы и ограничения AOSP-подхода	22
1.4. Российские мобильные ОС на базе Linux	23
1.4.1. Специфика Linux-based подхода	23
1.4.2. РОСА Мобайл	23
1.4.3. Аврора ОС	24
1.4.4. Сравнительная характеристика Linux-based ОС	28
1.4.5. Общие проблемы экосистемы	28
2. Применение мобильных ОС в корпоративной сфере	30
2.1. Концепции управления корпоративными устройствами	30
2.2. MDM и EMM	30
2.3. Требования к корпоративным ОС	32
2.4. Сравнение платформ для корпоративного использования	33
2.5. Российский корпоративный рынок	33
3. Применение мобильных ОС в атомной отрасли	35
3.1. Специфика отрасли и требования регуляторов	35
3.2. Мобилизация рабочих мест в атомной промышленности	36
3.3. Применение ОС «Аврора» в структурах Росатома	36
3.4. Требования к безопасности данных мобильных устройств	38
4. Виды мобильных приложений	40
4.1. Нативные приложения	40
4.1.1. Определение и принцип работы	40

4.1.2. Инструменты разработки	40
4.1.3. Преимущества и недостатки	42
4.2. Гибридные и кросс-платформенные приложения	42
4.2.1. Определение и мотивация	42
4.2.2. React Native	43
4.2.3. Flutter	44
4.2.4. Kotlin Multiplatform (KMP) и Compose Multiplatform (CMP) .	45
4.2.5. Xamarin и .NET MAUI	47
4.2.6. Сравнение кросс-платформенных подходов	48
4.2.7. Преимущества и компромиссы кросс-платформенного подхода	48
4.3. Прогрессивные веб-приложения (PWA)	49
4.3.1. Определение и концепция	49
4.3.2. Ключевые технологии	49
4.3.3. Ограничения на iOS и Android	52
4.3.4. Сценарии применения и ограничения в корпоративной среде	53
5. Стек технологий мобильной разработки	55
5.1. Языки и платформенные фреймворки	55
5.1.1. Kotlin	55
5.1.2. Swift	55
5.1.3. Dart	56
5.1.4. JavaScript / TypeScript	56
5.1.5. C++ и Qt/QML	56
5.2. Кросс-платформенные фреймворки: детальное сравнение	57
5.2.1. Flutter vs. React Native vs. KMP/CMP	57
5.3. KMP и ОС «Аврора»: Aurora Multiplatform	58
5.3.1. Архитектурная основа	58
5.3.2. Компоненты Aurora Multiplatform	58
5.3.3. Практическая схема KMP-проекта под «Аврору»	60
5.4. Backend и API	60
5.4.1. REST	60
5.4.2. GraphQL	60
5.4.3. gRPC	61
5.5. CI/CD для мобильных приложений	61
5.5.1. Инструменты сборки и автоматизации	61

5.5.2. Типовой CI/CD-пайплайн	62
5.6. Тестирование мобильных приложений	62
5.6.1. Пирамида тестирования	62
5.6.2. Unit-тестирование	62
5.6.3. UI-тестирование	63
5.6.4. E2E-тестирование	63
5.7. Безопасность мобильных приложений	63
5.7.1. Обфускация кода	63
5.7.2. Certificate Pinning	64
5.7.3. Хранение секретов	64
5.7.4. OWASP Mobile Top 10	64
6. Особенности Enterprise-приложений	66
6.1. Определение и отличие от consumer-приложений	66
6.2. Требования: надёжность, масштабируемость, интеграция	67
6.2.1. Надёжность и офлайн-режим	67
6.2.2. Масштабируемость	67
6.2.3. Интеграция с корпоративными системами	68
6.3. Архитектурные паттерны	69
6.3.1. Clean Architecture	69
6.3.2. MVVM и MVI	70
6.4. Управление корпоративными приложениями: MAM	71
6.5. Безопасность: шифрование, VPN, контейнеризация	71
6.5.1. Шифрование данных	71
6.5.2. VPN	72
6.5.3. Контейнеризация: Knox и Managed Profile	72
6.6. Примеры enterprise-приложений в атомной отрасли России	73
Заключение	74
Список использованных источников	76
Приложение А	80
Приложение Б	91

Введение

Стремительное развитие мобильных технологий за последние полтора десятилетия коренным образом изменило характер взаимодействия человека с информационными системами. Мобильные устройства — смартфоны и планшеты — из потребительского средства связи превратились в полноценный производственный инструмент, без которого сложно представить современное предприятие в любой отрасли. По данным аналитической компании Statista, к началу 2025 года мировое число активных мобильных устройств превысило 7,1 миллиарда единиц, а данные отчётов DataReportal подтверждают, что доля мобильного трафика в общем объёме интернет-трафика устойчиво держится выше 60% [1,2].

Ключевым фактором, определяющим функциональность и безопасность мобильного устройства, является операционная система. На глобальном рынке доминируют две платформы — Android компании Google и iOS компании Apple [3,4], однако в последние годы в России активно формируется третий сегмент: отечественные мобильные ОС, созданные в рамках государственной политики импортозамещения и программы «Цифровая экономика». Санкционное давление придало разработке суверенного программного обеспечения, включённого в Реестр отечественного ПО [5], характер стратегической необходимости. В числе приоритетных направлений — мобильная операционная система «Аврора» [6], сертифицированная ФСТЭК России для работы с конфиденциальной информацией.

Параллельно с эволюцией операционных систем усложняется и экосистема мобильных приложений. Современная классификация выделяет три принципиально различных подхода к разработке: нативные приложения; кросс-платформенные решения на базе современных фреймворков, таких как Flutter [7] и React Native [8]; и прогрессивные веб-приложения (PWA), стирающие границу между браузерным и нативным опытом. Каждый из этих подходов обладает специфическим набором компромиссов между производительностью, стоимостью разработки и возможностями интеграции с аппаратными ресурсами устройства.

Особый интерес представляет применение мобильных технологий в корпоративной среде и высокотехнологичных отраслях. Атомная промышленность является одним из наиболее показательных примеров: объекты атомной энергетики относятся к критической информационной инфраструктуре, что предопределяет использование защищенных решений в рамках стратегии циф-

ровой трансформации [9]. Требования к кибербезопасности систем в атомной отрасли регулируются в том числе международными стандартами МАГАТЭ.

Целью настоящего реферата является систематизированный анализ современного состояния мобильных операционных систем и приложений с акцентом на их применение в корпоративной и атомной отраслях.

Для достижения поставленной цели решаются следующие **задачи**:

- рассмотреть архитектуру и ключевые характеристики ведущих мобильных операционных систем: Android, iOS, а также российских разработок на базе AOSP и Linux;

- проанализировать особенности применения мобильных ОС в корпоративной среде, включая концепции управления устройствами и требования к информационной безопасности;

- изучить специфику внедрения мобильных технологий в атомной отрасли с учётом регуляторных ограничений и требований по защите информации;

- классифицировать виды мобильных приложений и охарактеризовать применяемые стеки технологий;

- выявить ключевые особенности разработки и эксплуатации корпоративных (enterprise) мобильных приложений.

Объектом исследования выступают мобильные операционные системы и программные приложения для них. **Предметом исследования** являются технологические, архитектурные и организационные особенности их применения в корпоративной и атомной отраслях.

Реферат состоит из двух частей. Первая часть посвящена мобильным операционным системам и охватывает три главы: обзор современных ОС, их применение в корпоративной сфере и специфику использования в атомной отрасли. Вторая часть посвящена мобильным приложениям и включает анализ видов приложений, стека технологий и особенностей enterprise-разработки.

1. Обзор современных мобильных ОС

1.1. Android

1.1.1. История и архитектура

Android — мобильная операционная система с открытым исходным кодом, разработанная компанией Android Inc., основанной в 2003 году Энди Рубином и приобретённой Google в 2005 году. Первый коммерческий релиз состоялся в 2008 году вместе с выходом смартфона HTC Dream. Открытый исходный код платформы распространяется в рамках проекта AOSP (Android Open Source Project) под лицензией Apache 2.0, что позволяет производителям устройств свободно модифицировать систему под собственные нужды.

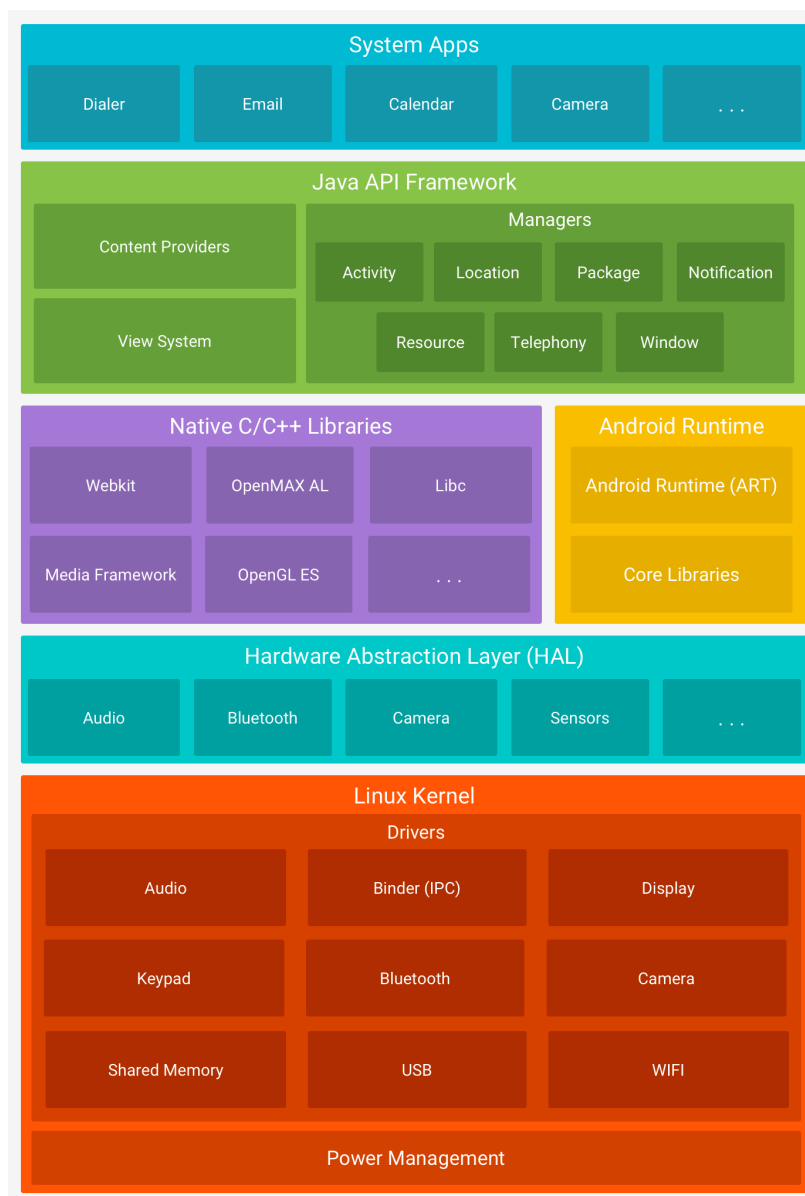


Рисунок 1.1 — Архитектура ОС Android[3]

Архитектура Android построена по многоуровневому принципу(см. Рис. 1.1):

- **Ядро Linux** (Linux Kernel) — нижний уровень стека; обеспечивает управление памятью, процессами, файловой системой, сетевым стеком и драйверами оборудования. Начиная с Android 4.0 используется ядро версии 3.x и выше;

- **Аппаратный абстрактный слой** (HAL, Hardware Abstraction Layer) — набор стандартизированных интерфейсов, позволяющих верхним уровням обращаться к аппаратным компонентам (камера, GPS, Bluetooth) независимо от реализации драйвера;

- **Android Runtime** (ART) — виртуальная машина, исполняющая байт-код приложений. Начиная с Android 5.0 полностью заменила Dalvik VM; использует AOT- (Ahead-of-Time) и JIT- (Just-in-Time) компиляцию для повышения производительности;

- **Нативные библиотеки** (Native Libraries) — набор C/C++ библиотек: OpenGL ES, SQLite, WebKit, libc и другие;

- **Java API Framework** — прикладные интерфейсы, предоставляемые разработчикам: менеджеры активностей, уведомлений, ресурсов, пакетов и оконной системы;

- **Слой приложений** (Applications) — пользовательские и системные приложения, работающие поверх фреймворка.

Начиная с Android 8.0 (Oreo, 2017) введена модульная архитектура **Project Treble**, разделяющая системный фреймворк и реализации HAL от производителей. Это позволило ускорить выпуск обновлений ОС производителями устройств. Дальнейшим развитием стал **Project Mainline** (Android 10+), позволяющий Google обновлять критически важные системные модули (Media, DNS, Security) напрямую через Google Play, минуя производителей.

1.1.2. Экосистема и доля рынка

Android является безусловно доминирующей мобильной платформой в мире. По данным StatCounter, в 2025 году доля Android на рынке мобильных операционных систем составляет около 72% [10]. В развивающихся рынках — Азии, Африки, Латинской Америки — этот показатель существенно выше и достигает 85–90%.

Экосистема Android охватывает несколько взаимосвязанных компонентов. **Google Play** является основным магазином приложений с каталогом более 3 миллионов приложений. Наряду с ним существуют альтернативные маркет-

плейсы: Amazon Appstore, Samsung Galaxy Store, а также российские магазины — RuStore и NashStore, созданные после ограничения работы Google Play на территории России в 2022 году. **Google Mobile Services (GMS)** — закрытый набор сервисов и API (Google Maps, Push-уведомления через FCM¹, авторизация через Google Account), предустанавливаемый на устройства по лицензионному соглашению с Google. Устройства без GMS — в частности, смартфоны Huawei после 2019 года — вынуждены использовать альтернативные реализации или собственные сервисы (HMS²).

Широчайший спектр аппаратных партнёров (Samsung, Xiaomi, OPPO, Vivo, Motorola, Sony и сотни других) обеспечивает Android присутствие в ценовых сегментах от бюджетных устройств до флагманских смартфонов.

1.1.3. Версии, фрагментация и модель обновлений

С 2008 года выпущено более 20 мажорных версий Android. Начиная с Android 10 Google отказалась от традиции именования версий в честь сладостей (Cupcake, Donut, KitKat...) и перешла на числовую нумерацию.



Рисунок 1.2 — Распределение версий Android (1 декабря 2025)³

¹Firebase Cloud Messaging

²Huawei Mobile Services

³Источник: <https://composables.com/android-distribution-chart>

Ключевой проблемой экосистемы Android остаётся **фрагментация** (см. Рис. 1.2) — неравномерное распределение устройств по версиям ОС. В отличие от iOS, где обновления распространяются централизованно, Android-обновления проходят через несколько звеньев цепочки:

1. Google выпускает новую версию AOSP и патчи безопасности;
2. Производитель устройства (OEM) адаптирует версию под своё «железо» и программную оболочку (One UI у Samsung, MIUI у Xiaomi и т.д.);
3. Оператор сотовой связи (в ряде регионов) проводит собственное тестирование перед распространением.

Каждое звено добавляет задержку, которая в среднем составляет от нескольких недель до нескольких месяцев. Устройства нижнего ценового сегмента нередко получают лишь одно-два мажорных обновления за весь жизненный цикл. Частичным решением служит политика **Android Enterprise Recommended** — программа сертификации устройств, обязывающая производителей гарантировать минимум три года обновлений безопасности.

1.1.4. Безопасность и управление устройствами (MDM/EMM)

Архитектура безопасности Android реализует принцип **многоуровневой защиты** (defence in depth) [11]:

— **Sandbox-изоляция** — каждое приложение работает в изолированном процессе с уникальным UID, что ограничивает доступ к ресурсам других приложений;

— **Система разрешений** (Permissions) — начиная с Android 6.0 разрешения на доступ к чувствительным ресурсам (местоположение, камера, контакты) запрашиваются во время выполнения (runtime), а не при установке;

— **SELinux** (Security-Enhanced Linux) — обязательный контроль доступа (MAC), принудительно включён начиная с Android 5.0;

— **Verified Boot / dm-verity** — механизм проверки целостности загрузчика и системного раздела при каждом старте устройства;

— **Google Play Protect** — встроенный антивирусный модуль, ежедневно сканирующий более 125 миллиардов приложений [11].

Для корпоративного управления Android с версии 5.0 предоставляет платформу **Android Enterprise** (ранее Android for Work). Она определяет три основных сценария развёртывания: BYOD⁴ (личное устройство с рабочим профилем), COPE⁵ (корпоративное устройство с личным профилем) и COBO⁶

⁴Bring Your Own Device

⁵Corporate-Owned, Personally Enabled

(полностью корпоративное устройство в режиме киоска)[12]. Управление реализуется через **MDM-агент** (Mobile Device Management), взаимодействующий с Device Policy Controller API. На российском рынке среди MDM-решений распространены: Kaspersky Endpoint Security, «Мобильный контроль» от «ИнфоТеКС», а также зарубежные VMware Workspace ONE и Microsoft Intune.

1.2. iOS

1.2.1. История и архитектура

iOS — мобильная операционная система компании Apple Inc., представленная в январе 2007 года вместе с первым поколением iPhone. Изначально система не имела собственного имени и позиционировалась как «OS X, который работает на телефоне»; торговое наименование iOS закрепилось лишь с 2010 года. Сегодня iOS является основой всего семейства мобильных платформ Apple: iPadOS, tvOS, watchOS и visionOS построены на её общем ядре и разделяют значительную часть системных компонентов.

⁶Corporate-Owned, Business Only

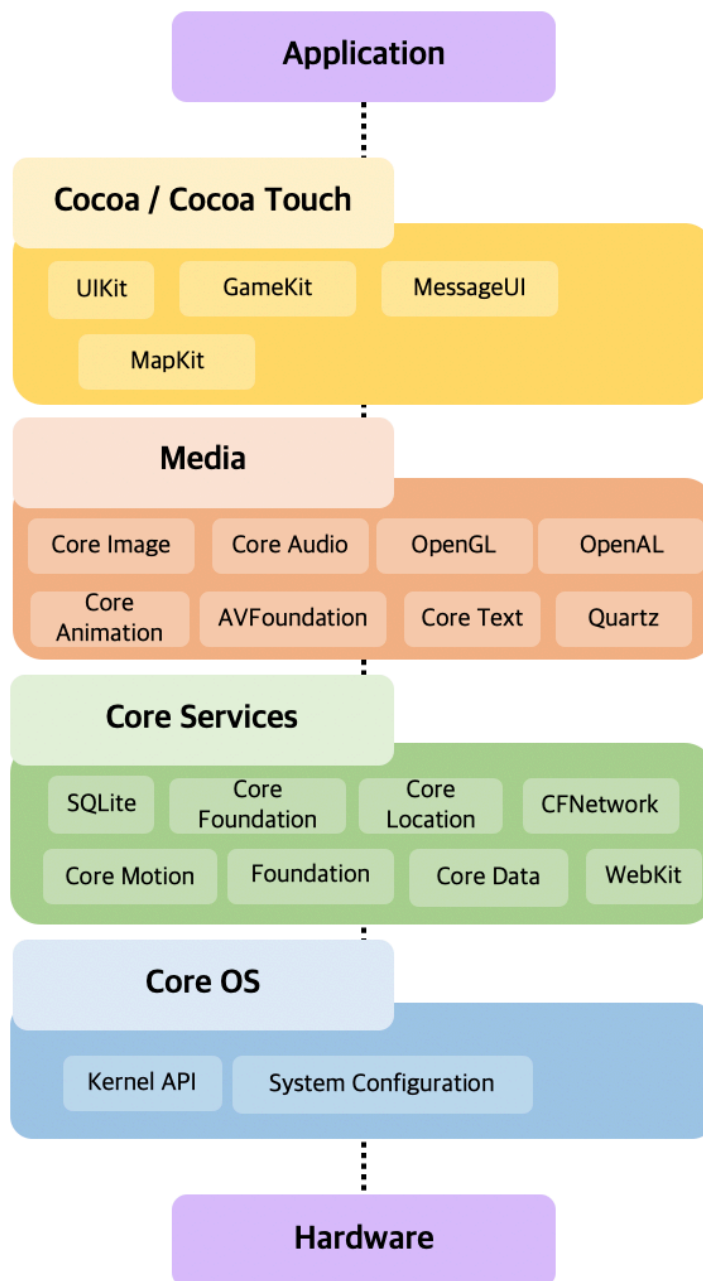


Рисунок 1.3 — Архитектура iOS[4]

В основе iOS лежит микроядро **XNU** (X is Not Unix) — гибридное ядро, объединяющее компоненты Mach и BSD. Архитектура системы организована в четыре уровня абстракции (см. Рис. 1.3):

- **Core OS** — нижний уровень: ядро XNU, драйверы, криптографическая подсистема (Secure Enclave Processor), Bluetooth и Wi-Fi-стек, управление питанием;

- **Core Services** — фундаментальные фреймворки: Foundation, Core Data, Core Location, CloudKit; именно на этом уровне реализованы базовые сервисы для работы приложений;

— **Media** — графические, аудио- и видеотехнологии: Metal (низкоуровневый GPU API), Core Graphics, AVFoundation, Core Animation;

— **Cocoa Touch** — верхний уровень, непосредственно используемый разработчиками приложений: UIKit, SwiftUI, MapKit, Push-уведомления через APNs (Apple Push Notification service).

Единственным официальным языком прикладной разработки для iOS является **Swift**, представленный Apple в 2014 году и постепенно вытеснивший Objective-C. Swift обеспечивает статическую типизацию, безопасную работу с памятью без ручного управления и высокую производительность, сопоставимую с C++.

1.2.2. Закрытая экосистема Apple: плюсы и минусы

Фундаментальное отличие iOS от Android — **полностью закрытая вертикальная экосистема**. Apple контролирует весь стек: аппаратное обеспечение (собственные чипы серии A и M), операционную систему, среду разработки (Xcode), магазин приложений (App Store) и платёжную инфраструктуру (Apple Pay). Такая модель имеет выраженные преимущества и ограничения.

Преимущества закрытой модели:

— **Единообразие обновлений**. Apple централизованно распространяет обновления iOS напрямую на устройства, минуя производителей и операторов. Новая версия ОС в течение первых недель охватывает более 70% активных устройств — показатель, принципиально недостижимый в экосистеме Android;

— **Контроль качества App Store**. Каждое приложение проходит ручную модерацию перед публикацией, что существенно снижает риск распространения вредоносного ПО через официальный канал;

— **Оптимизация под «железо»**. Совместная разработка чипа и ОС позволяет Apple добиваться исключительной энергоэффективности и производительности;

— **Долгий цикл поддержки**. Apple предоставляет обновления ПО в течение 5–7 лет с момента выпуска устройства.

Ограничения закрытой модели:

— **Высокая стоимость устройств** ограничивает распространение iOS в развивающихся рынках и бюджетном корпоративном сегменте;

— **Отсутствие альтернативных магазинов приложений** (до принятия в ЕС Digital Markets Act в 2024 году) ограничивало конкуренцию и создавало зависимость разработчиков от политики Apple;

— **Закрытость API** ряда системных компонентов исторически ограничивала возможности сторонних разработчиков по сравнению с нативными приложениями Apple;

— **Невозможность глубокой кастомизации** системы делает iOS непригодной для создания специализированных промышленных форм-факторов и встроенных решений на её основе.

1.2.3. Политика безопасности и конфиденциальности

Безопасность является одним из ключевых конкурентных преимуществ iOS. Система реализует принцип «**безопасность по умолчанию**» на нескольких уровнях [4]:

Аппаратный уровень. Все современные устройства Apple оснащены **Secure Enclave** — изолированным сопроцессором, физически отделённым от основного CPU. Secure Enclave хранит криптографические ключи (в том числе биометрические данные Face ID и Touch ID) и никогда не передаёт их в основную систему — даже при компрометации ОС.

Уровень загрузки. Цепочка доверенной загрузки (Secure Boot Chain) проверяет каждый компонент последовательно: загрузчик, ядро, расширения ядра, системные компоненты. Подпись каждого этапа верифицируется аппаратными ключами, зашитыми в чип.

Уровень приложений. Каждое приложение выполняется в изолированной **sandbox**-среде и лишено доступа к файлам, данным и ресурсам других приложений без явного пользовательского разрешения. Начиная с iOS 14 система визуально оповещает пользователя о каждом обращении приложения к камере, микрофону или буферу обмена.

Конфиденциальность. Apple последовательно позиционирует конфиденциальность как «базовое право человека». Ключевые механизмы: App Tracking Transparency (ATT) — с iOS 14.5 каждое приложение обязано явно запросить согласие на межсайтовое отслеживание; Private Relay — двухузловой прокси для Safari, скрывающий IP-адрес и DNS-запросы; Hide My Email — генерация одноразовых адресов электронной почты.

1.2.4. Корпоративный профиль и Apple Business Manager

Apple целенаправленно развивает корпоративное направление с 2014 года, когда была запущена программа **Apple Business Manager** (ABM) — единый веб-портал для централизованного управления устройствами, приложениями и аккаунтами сотрудников организации.

Ключевые корпоративные возможности платформы:

Device Enrollment Program (DEP) / Automated Device Enrollment — механизм «нулевого касания» (zero-touch deployment): устройство, приобретённое через АВМ, при первом включении автоматически регистрируется в корпоративной MDM-системе без участия IT-специалиста. Это критически важно для масштабного развёртывания в крупных организациях.

Managed Apple ID — корпоративные идентификаторы, контролируемые IT-отделом организации, отделённые от личных Apple ID сотрудников.

Volume Purchase Program (VPP) — корпоративная закупка и централизованное распределение лицензий на приложения без необходимости использования личных Apple ID сотрудников.

Управляемые конфигурации — MDM-система может принудительно устанавливать политики: требования к длине пароля, запрет на использование камеры, принудительное шифрование, настройку VPN и Wi-Fi. Конфигурационные профили распространяются в формате XML (`.mobileconfig`).

Совместимость iOS с ведущими MDM-платформами — VMware Workspace ONE, Microsoft Intune, Jamf Pro — обеспечивает бесшовную интеграцию в корпоративную инфраструктуру. В российских реалиях после 2022 года часть международных MDM-решений оказалась недоступна, что усилило интерес к отечественным альтернативам и к платформе «Аврора» как замене iOS в чувствительных сегментах.

1.3. Российские мобильные ОС на базе AOSP

1.3.1. Предпосылки создания отечественных ОС

Разработка российских мобильных операционных систем приобрела стратегический характер под влиянием двух взаимосвязанных факторов.

Первый — государственная политика импортозамещения в сфере программного обеспечения. Начиная с 2015 года Министерство цифрового развития ведёт Единый реестр российского ПО [5], включение в который стало обязательным условием для участия отечественных разработчиков в государственных закупках. Мобильные операционные системы как класс ПО оказались в числе приоритетных направлений реестра.

Второй фактор — санкционное давление. После 2022 года ряд российских компаний лишился доступа к Google Mobile Services: лицензии GMS не могут быть выданы организациям, находящимся под блокирующими санкциями США. В частности, компания YADRO (ООО «КНС Групп»), попавшая под

санкции в феврале 2022 года, была вынуждена выстраивать экосистему без какого-либо участия Google.

В этих условиях наиболее практичной технической стратегией стала разработка на базе **AOSP** (Android Open Source Project). AOSP распространяется под лицензией Apache 2.0 и не требует лицензионных соглашений с Google, при этом сохраняя совместимость с Android-приложениями. Принципиальный компромисс такого подхода: без лицензии **GMS** разработчик не может легально использовать официальный Android SDK от Google и лишается экосистемы сервисов (карты, push-уведомления через FCM, Play Market). Их место занимают российские альтернативы — прежде всего магазин приложений RuStore, запущенный VK в 2022 году по поручению Правительства РФ.

1.3.2. KVADRA_OS

KVADRA_OS — мобильная операционная система компании YADRO (бренд клиентских устройств Kvadra), публично представленная в мае 2024 года [13]. ОС создана на базе AOSP и включает компоненты собственной разработки, а также переработанные open source решения: собственное ядро, системные приложения и сервисы.

Флагманским устройством экосистемы стал планшет **Kvadra_T** с диагональю экрана 10,95 дюйма, поступивший в продажу в апреле 2024 года по цене около 42 000 рублей. Согласно заявлениям компании, производственные мощности позволяют выпускать до 1 миллиона планшетов в год, что ставит **KVADRA_OS** на позицию потенциально наиболее массовой отечественной мобильной ОС.

Ключевые технические и продуктовые особенности системы:

- замена **GMS** на собственные сервисы Kvadra: облачное хранилище (15 ГБ в базовой комплектации), резервное копирование, сервисный центр «Центр»;
- магазин приложений — RuStore;
- многопользовательский режим с изолированными учётными записями для разных пользователей устройства;
- **KVADRA_OS** внесена в Единый реестр российского ПО 26 декабря 2023 года.

Отдельно компания анонсировала корпоративную конфигурацию устройств для государственного сектора под управлением ОС «Аврора», признавая, что AOSP-версия ориентирована преимущественно на потреби-

тельный рынок, тогда как режимные объекты требуют более строгой сертификации.

1.3.3. РЕД ОС М

РЕД ОС М — мобильная операционная система компании «РЕД СОФТ», включённая в Реестр российского ПО 18 октября 2023 года (запись № 19638) [14]. Система совместима как с Android-, так и с Linux-приложениями, что является её ключевой особенностью.

РЕД ОС М предлагает два режима работы:

— **Мобильный режим** — классический интерфейс смартфона или планшета с привычными жестами, галереей, файловым менеджером, камерой и магазином приложений RuStore. Поддерживаются обновления «по воздуху» (OTA⁷);

— **Настольный режим** — при подключении монитора, клавиатуры и мыши устройство превращается в рабочую станцию с рабочим столом, аналогичным РЕД ОС (настольный Linux-дистрибутив компании «РЕД СОФТ»).

С точки зрения информационной безопасности система изначально проектировалась без зависимости от облачных сервисов Google: геолокация реализована через отечественную систему «А-ГЛОНАСС», синхронизация времени использует российские NTP-серверы⁸, сборка всех компонентов ОС производится на мощностях «РЕД СОФТ». Система поддерживает российское криптографическое программное обеспечение (ГОСТ-алгоритмы).

1.3.4. Общие проблемы и ограничения AOSP-подхода

Несмотря на привлекательность AOSP как стартовой платформы, этот подход сопряжён с рядом системных ограничений.

Юридические риски. Android SDK — официальный инструментарий разработки приложений — распространяется Google по лицензии, несовместимой с использованием на глубоко модифицированных форках AOSP. Это означает, что разработчики, пишущие приложения для AOSP-систем, технически обязаны использовать только открытые инструменты (AOSP build tools), что существенно сужает доступную экосистему разработки.

Отставание от апстрима. Google регулярно выпускает обновления безопасности и новые версии AOSP. Российские форки вынуждены самосто-

⁷Over The Air

⁸Network Time Protocol

ятельно портировать эти изменения, что требует значительных инженерных ресурсов и неизбежно создаёт временной лаг.

Приложения и экосистема. Отсутствие Google Play лишает пользователей доступа к миллионам привычных приложений. Российский RuStore активно наращивает каталог, однако по состоянию на 2025 год существенно уступает Play Market по числу приложений и их качеству, особенно в категориях корпоративного ПО.

Вопрос суверенности. Часть экспертного сообщества ставит под сомнение подлинную «отечественность» AOSP-систем, указывая на то, что ядро платформы по-прежнему разрабатывается и контролируется Google. Этот дискуссионный вопрос публично поднимался при рассмотрении заявок на включение в реестр Минцифры.

1.4. Российские мобильные ОС на базе Linux

1.4.1. Специфика Linux-based подхода

Принципиальное отличие Linux-based мобильных ОС от решений на базе AOSP — в фундаменте платформы. AOSP, несмотря на использование ядра Linux, представляет собой самостоятельный технологический стек (ART, Binder IPC, HAL), созданный и контролируемый Google. Linux-based ОС в мобильном контексте опирается на классическое ядро Linux в связке с открытыми пользовательскими средами (Qt, KDE Plasma Mobile, Wayland) — без какой-либо зависимости от Google на архитектурном уровне.

Это даёт ключевое преимущество с точки зрения цифрового суверенитета: кодовая база не связана ни с одной американской корпорацией, а разработка ведётся поверх многолетней международной open source экосистемы, где Россия может выступать полноправным контрибьютором. Однако это же обуславливает главный недостаток — полное отсутствие нативной совместимости с Android-приложениями. Её приходится реализовывать через отдельный уровень эмуляции, что неизбежно сказывается на производительности и степени интеграции.

1.4.2. РОСА Мобайл

РОСА Мобайл — мобильная операционная система компании «НТЦ ИТ РОСА», построенная на ядре Linux с графической оболочкой KDE Plasma Mobile. Система является логичным продолжением линейки десктопных ОС РОСА Хром и опирается на собственный репозиторий ROSA 2021.1, формировавшийся более 10 лет и насчитывающий свыше 30 000 пакетов [15].

Публично система была представлена в декабре 2023 года совместно со смартфоном **Р-ФОН** производства АО «Рутек». Внесена в Реестр российского ПО Минцифры (запись № 16453 от 03.02.2023), а в ноябре 2024 года получила сертификат ФСТЭК России № 4867 (4-й уровень доверия, действителен до 13.11.2029) [15].

Ключевые технические особенности:

- собственные драйверы для каждого аппаратного компонента (камера, микрофон, Bluetooth, Wi-Fi, GNSS), написанные инженерами РОСА без использования закрытых бинарных блобов;
- встроенный эмулятор для запуска APK-пакетов (Android-приложений) при условии соответствия политике безопасности организации;
- собственная инфраструктура push-уведомлений, независимая от FCM (Google);
- единая экосистема РОСА ID с облачным хранилищем, мессенджером и почтовым клиентом;
- система управления корпоративной мобильностью «РОСА Контроль» (MDM-функциональность: удалённое управление устройствами, контроль доступа к сети и периферии, определение местоположения).

1.4.3. Аврора ОС

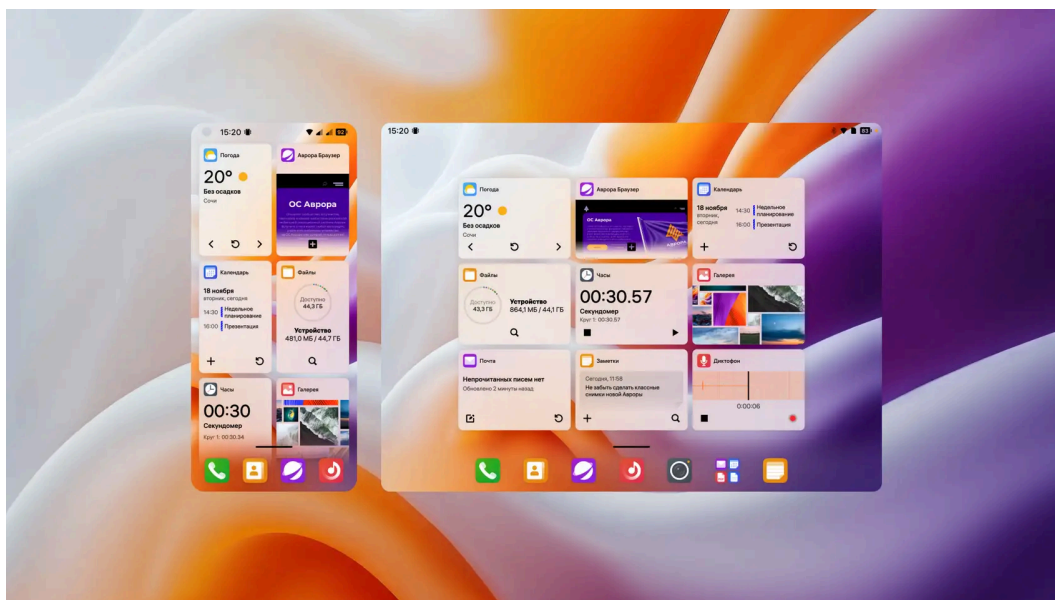


Рисунок 1.4 — Интерфейс Аврора ОС 5.2

Аврора ОС — флагманская российская мобильная операционная система, разработанная ООО «Открытая мобильная платформа» (ОМП) [16]. С технической точки зрения система основана на **Sailfish OS** финской компании

Jolla, которая в свою очередь является наследником MeeGo — совместного проекта Nokia и Intel, закрытого в 2011 году. Лицензионное соглашение между ОМП и Jolla заключено в 2016 году; до 2019 года система была известна под именем Sailfish Mobile OS RUS.

1.4.3.1. Архитектура

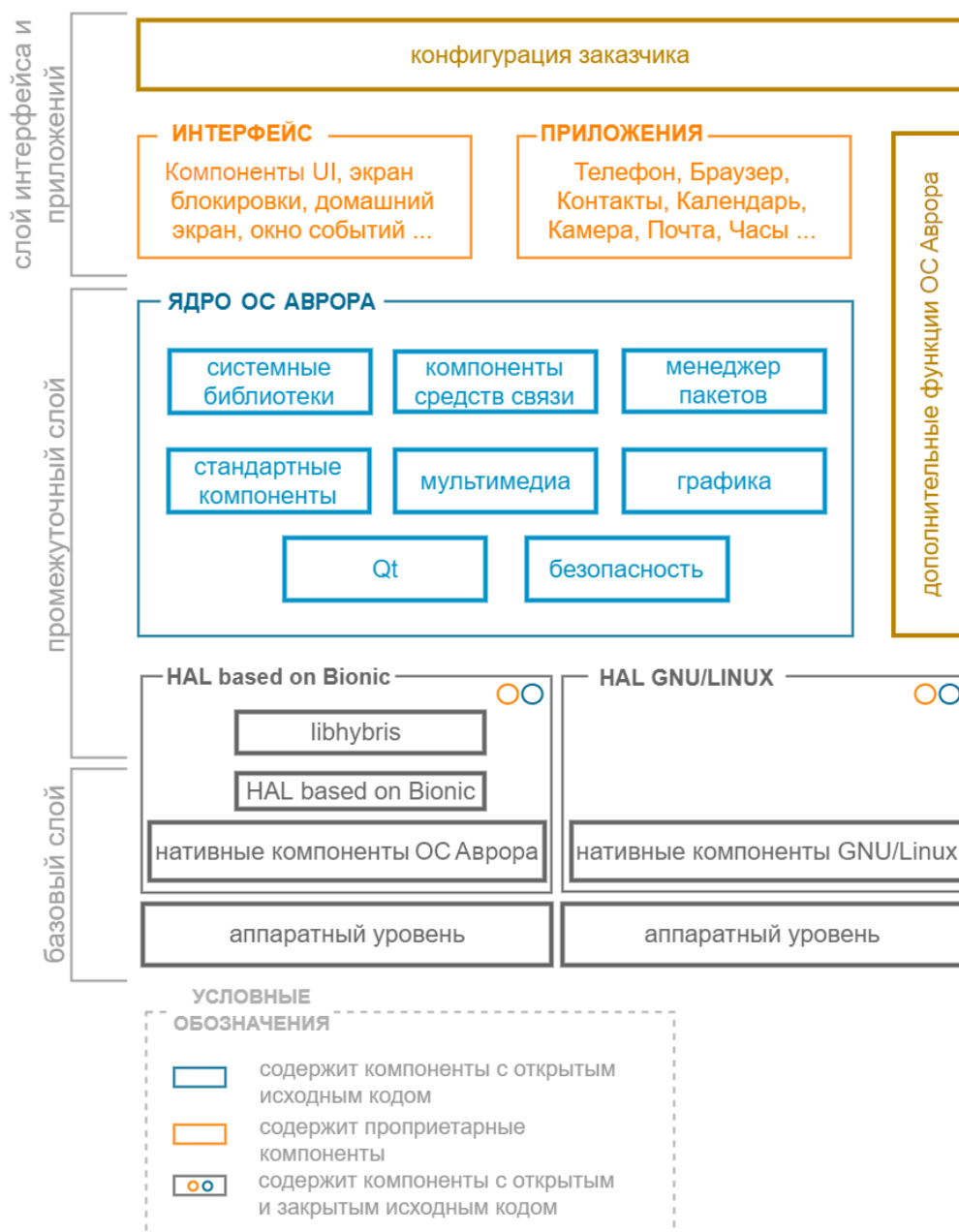


Рисунок 1.5 — Архитектура ОС Аврора⁹

Архитектурный стек «Авроры» принципиально отличается от Android (см. Рис. 1.5):

— **Ядро**: стандартное ядро Linux;

⁹Источник: <https://developer.auroraos.ru/doc/platform/architecture>

— **Графическая подсистема:** Wayland (комpositor Lipstick) вместо SurfaceFlinger;

— **Языки разработки приложений:** C++ (Qt/QML) — основной; Python, Rust поддерживаются через соответствующие привязки; Java не поддерживается (отсутствие JVM¹⁰); Есть возможность использования Kotlin Multiplatform (Compose Multiplatform в Alpha-версии);

— **Пакетный формат:** RPM, управляемый пакетным менеджером `pkcon/zypper` ;

— **MDM-платформа:** «Аврора Центр» — собственная EMM-система ОМП, обеспечивающая управление устройствами, приложениями и политиками безопасности через защищённый канал [17].

SDK Аврора (Aurora SDK) включён в Реестр российского ПО и поддерживает компиляцию под архитектуры ARM32, AArch64 и x86_64, включая российские процессоры. Среда разработки основана на Qt Creator с плагином для Аврора, сборка производится в изолированном Linux-окружении (VirtualBox-образ), что обеспечивает воспроизводимые билды [18].

1.4.3.2. Сертификация и безопасность

«Аврора» — единственная российская мобильная ОС с комплексной двойной сертификацией регуляторов:

— **ФСТЭК России:** сертификат по профилю защиты ОС типа «А» 4-го класса (ИТ.ОС.А4.ПЗ), уровень доверия УД4, а также соответствие НДВ4; позволяет применять систему на объектах КИИ 1-й категории, в ГИС 1-го класса и АСУ ТП 1-го класса защищённости;

— **ФСБ России:** сертификат классов АКЗ/КСЗ (апрель 2024 года), полученный совместно со СКЗИ «СледопытSSL»; открывает применение в органах высшей государственной власти и финансовой сфере.

К встроенным механизмам безопасности относятся: гарантированное удаление данных, контроль целостности файловой системы, верификация цифровой подписи пакетов при установке, ограничение программной среды (запрет установки ПО из недоверенных источников), TEE¹¹ (см. Рис. 1.6).

¹⁰Java Virtual Machine

¹¹TEE (Trusted Execution Environment) — аппаратно изолированная среда выполнения в составе процессора (ARM TrustZone). Обеспечивает доверенное хранение ключей и выполнение криптографических операций, физически изолированное от основной ОС.

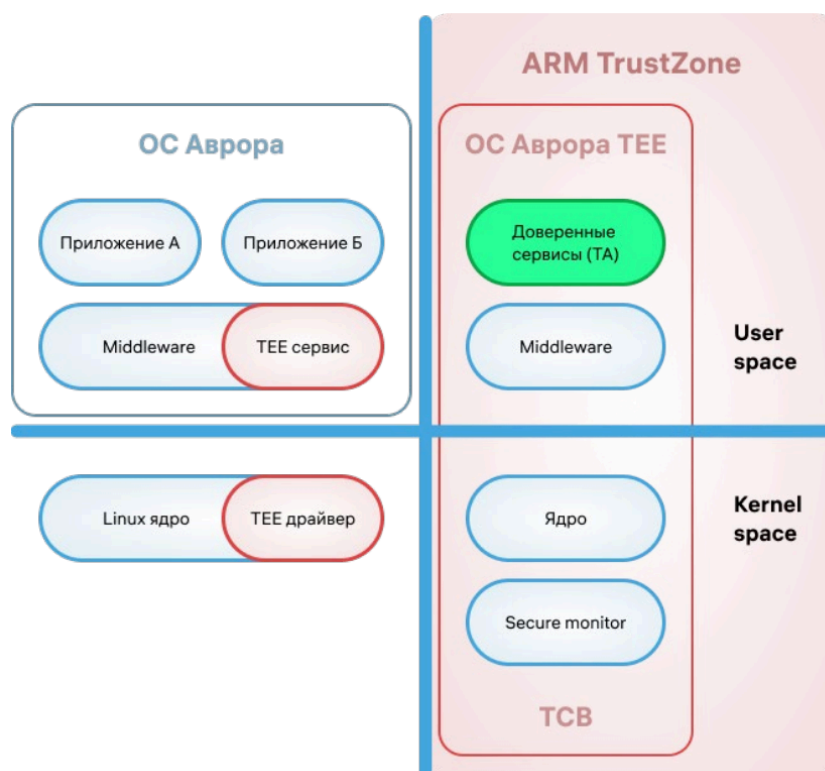


Рисунок 1.6 — TEE ОС Аврора

1.4.3.3. Применение

Основная целевая аудитория «Авроры» — государственные органы и крупные корпорации с повышенными требованиями к информационной безопасности. По состоянию на начало 2024 года число развёрнутых устройств достигло 500 тысяч; в розничной продаже реализовано около 800 смартфонов и 130 планшетов. В октябре 2024 года ОМП объявила программу «Аврора+», предусматривающую портирование ОС на телевизоры, роутеры и бортовую автоэлектронику [16].

1.4.4. Сравнительная характеристика Linux-based ОС

Таблица 1.1 — Сравнение российских мобильных ОС на базе Linux

Характеристика	Аврора ОС	РОСА Мобайл
Разработчик	ООО «Открытая мобильная платформа»	АО «НТЦ ИТ РОСА»
Основа	Sailfish OS / ядро Linux	Ядро Linux + KDE Plasma Mobile
Год первого релиза	2016 (как Sailfish Mobile OS RUS)	2023
Язык разработки приложений	C++ / Qt / QML	Qt / QML, поддержка APK-эмуляции
Пакетный формат	RPM	RPM
Android-совместимость	Отсутствует (нативно)	Через встроенный эмулятор
MDM-платформа	«Аврора Центр» (EMM)	«РОСА Контроль»
Сертификат ФСТЭК	УД4, профиль А4	УД4, профиль А4 (№ 4867)
Сертификат ФСБ	АК3/КС3 (2024)	Нет данных
Реестр Минцифры	Да	Да (запись № 16453)
Магазин приложений	Аврора Маркет / RuStore	РОСА Маркет
Целевой сегмент	Госсектор, КИИ, корпорации	Бизнес, госсектор, потребительский
Push-уведомления	Собственная инфраструктура	Собственная инфраструктура

1.4.5. Общие проблемы экосистемы

Несмотря на значительный прогресс, Linux-based мобильные ОС сталкиваются с рядом системных барьеров.

Экосистема приложений остаётся главным ограничением. Каталоги Аврора Маркет и РОСА Маркет несопоставимо меньше Google Play, а разработка под Qt/QML требует от команд переработки существующих Android- и iOS-приложений практически с нуля — в отличие от AOSP-систем, где совместимость с Android APK достигается автоматически.

Инструментарий разработки. Экосистема Qt зрелая и кросс-платформенная, однако значительно менее распространена среди российских мобильных разработчиков, чем Kotlin или Flutter. Это увеличивает стоимость и время разработки корпоративных приложений.

Аппаратная поддержка. Количество устройств, сертифицированных для работы с «Авророй» и РОСА Мобайл, существенно ограничено по сравнению с широким парком Android-устройств. Это создаёт зависимость от узкого круга партнёров-производителей оборудования.

2. Применение мобильных ОС в корпоративной сфере

2.1. Концепции управления корпоративными устройствами

В корпоративной практике сложились три базовые модели владения мобильными устройствами, определяющие политику безопасности и управляемости:

Таблица 2.1 — Модели владения мобильными устройствами

Параметр	BYOD	COPE	COBO
Расшифровка	Bring Your Own Device	Corporate-Owned, Personally Enabled	Corporate-Owned, Business Only
Владелец устройства	Сотрудник	Организация	Организация
Личное использование	Да	Да (ограниченно)	Нет
Контроль ИТ-отдела	Частичный	Полный	Полный
Типичный сценарий	Офисные сотрудники	Менеджеры среднего звена	Промышленность, госсектор
Стоимость для компании	Низкая	Средняя	Высокая

В российском корпоративном секторе наблюдается смещение от BYOD к COPE и COBO: режимные предприятия и государственные органы требуют полного контроля над устройством, что делает модели BYOD неприемлемыми по соображениям безопасности.

2.2. MDM и EMM

Mobile Device Management (MDM) — базовый уровень управления: инвентаризация устройств, удалённая блокировка и очистка, настройка Wi-Fi и VPN. **Enterprise Mobility Management (EMM)** расширяет MDM, добавляя управление приложениями (MAM) и контентом (MCM) (см. Рис. 2.1).

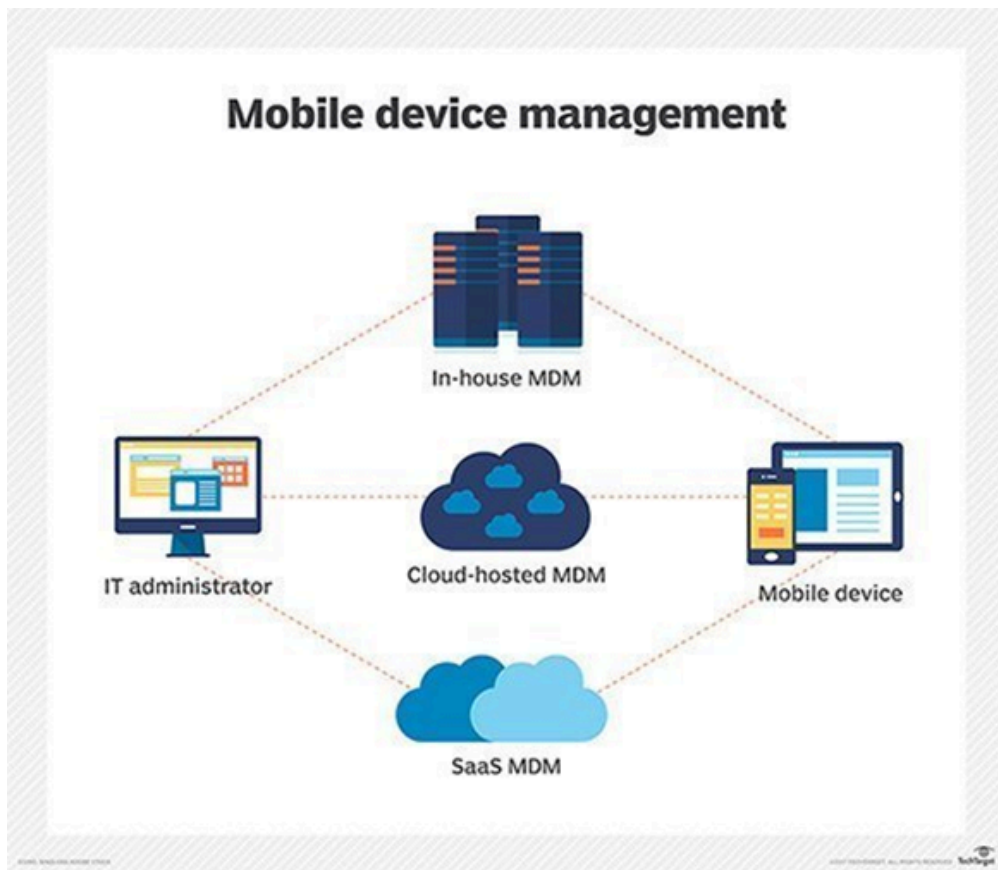


Рисунок 2.1 — Диаграмма архитектуры MDM

Типовые функции EMM-платформы:

— принудительное применение политик паролей и шифрования; — изолированный корпоративный контейнер (отделяет рабочие данные от личных на устройстве); — удалённое стирание только корпоративного контейнера (selective wipe); — управление жизненным циклом приложений — установка, обновление, отзыв; — geo-fencing и мониторинг соответствия (compliance).

2.3. Требования к корпоративным ОС

Корпоративное применение предъявляет к мобильной ОС требования, принципиально отличающиеся от потребительских:

Таблица 2.2 — Ключевые требования к корпоративным мобильным ОС

Область	Требования
Безопасность	Аппаратное шифрование, Secure Boot, изоляция контейнеров, поддержка ГОСТ-криптографии (для РФ), сертификаты ФСТЭК/ФСБ
Управляемость	Device Policy Controller API, поддержка MDM/EMM, zero-touch enrollment, OTA-обновления под контролем ИТ
Интеграция	LDAP/AD, корпоративный PKI, VPN (IKEv2, WireGuard), SSO через SAML/OIDC
Надёжность	Долгосрочная поддержка (LTS), предсказуемый цикл обновлений, гарантия патчей безопасности
Аудит	Журналирование событий, интеграция с SIEM, соответствие 152-ФЗ и требованиям КИИ (187-ФЗ) для РФ

2.4. Сравнение платформ для корпоративного использования

Таблица 2.3 — Сравнение мобильных платформ для корпоративного применения

Критерий	Android Enterprise	iOS / iPadOS	Аврора ОС
Zero-touch enrollment	Да	Да (ABM/DEP)	Да (Аврора Центр)
Рабочий профиль / контейнер	Managed Profile	Managed Open In	Корпоративный профиль
MDM API	Device Policy Controller	MDM Protocol (Apple)	Аврора Центр EMM
Поддержка ГОСТ-крипто	Сторонние решения	Сторонние решения	Встроена (ФСБ КСЗ)
Сертификат ФСТЭК	Нет	Нет	УД4, профиль А4
Гарантия обновлений	3–5 лет (AER)	5–7 лет	По договору с ОМП
Магазин приложений	Google Play / RuStore	App Store	Аврора Маркет
Кастомизация ОС	Высокая (AOSP)	Минимальная	Средняя
Зависимость от иностранного вендора	Google (GMS)	Apple	Нет
Стоимость лицензии ОС	Бесплатно	Включена в устройство	Коммерческая лицензия

Принципиальное конкурентное преимущество «Авроры» в корпоративном контексте — полное отсутствие зависимости от иностранных вендоров и наличие российских регуляторных сертификатов. Слабая сторона — существенно меньший каталог прикладного ПО и более узкий выбор аппаратных платформ.

2.5. Российский корпоративный рынок

После 2022 года российский рынок корпоративной мобильности претерпел структурные изменения. Ряд зарубежных EMM-платформ (VMware

Workspace ONE, Microsoft Intune) оказался недоступен для отечественных заказчиков, что стимулировало развитие российских MDM-решений.

Среди актуальных отечественных продуктов выделяются:

— **«Аврора Центр»** (ОМП) — EMM-платформа для устройств под управлением ОС «Аврора», сертифицированная ФСТЭК по УД4 [17];

— **SafeMobile UEM** (компания «МобилитиЛаб») — универсальная платформа для управления Android и iOS в российских компаниях;

— **Kaspersky Endpoint Security for Mobile** — MDM-модуль в составе платформы Kaspersky Security Center, работающий с Android и iOS;

— **«Мобильный контроль»** (ИнфоТеКС) — MDM-решение с поддержкой ГОСТ-шифрования для организаций, обрабатывающих конфиденциальную информацию.

Ключевой тренд — смещение государственных заказчиков и предприятий критической инфраструктуры в сторону устройств под управлением ОС «Аврора» как единственной отечественной платформы с полным комплектом регуляторных сертификатов. Коммерческий сектор в большинстве случаев продолжает работу на Android с российским MDM, рассматривая переход как долгосрочную перспективу по мере расширения экосистемы отечественных приложений.

3. Применение мобильных ОС в атомной отрасли

3.1. Специфика отрасли и требования регуляторов

Атомная промышленность формирует один из наиболее жёстких регуляторных контекстов для применения информационных технологий. Ядерные объекты — АЭС¹², предприятия ядерного топливного цикла, исследовательские реакторы — отнесены к критической информационной инфраструктуре (КИИ)¹³ первой категории значимости, что предопределяет особые требования к любому программному и аппаратному обеспечению, эксплуатируемому на этих объектах.

Основными регуляторами в части информационной безопасности на объектах атомной отрасли выступают:

— **Госкорпорация «Росатом»** — отраслевой регулятор, устанавливающий корпоративные стандарты информационной безопасности в рамках единой цифровой политики;

— **ФСТЭК России**¹⁴ — регулятор в сфере технической защиты информации и безопасности КИИ; устанавливает требования к применяемым средствам защиты через реестр сертифицированных СЗИ¹⁵

— **ФСБ России** — регулятор в области криптографической защиты информации; обязательна поддержка отечественных ГОСТ-алгоритмов шифрования;

— **МАГАТЭ**¹⁶ — в части международных рекомендаций по компьютерной безопасности АСУ ТП¹⁷ атомных станций [19].

Ключевое следствие для мобильных устройств: любой гаджет, используемый персоналом на режимном ядерном объекте, обязан работать под управлением ОС, сертифицированной ФСТЭК и ФСБ. На практике это означает, что применение Android и iOS на объектах КИИ первой категории не допускается — ни одна из этих систем не прошла соответствующую сертификацию.

¹² АЭС — атомная электростанция.

¹³ КИИ — критическая информационная инфраструктура. Правовое определение закреплено Федеральным законом № 187-ФЗ от 26.07.2017 «О безопасности КИИ Российской Федерации».

¹⁴ ФСТЭК — Федеральная служба по техническому и экспортному контролю. Регулирует защиту информации, не содержащей сведений, составляющих государственную тайну, а также безопасность КИИ.

¹⁵ СЗИ — средство защиты информации.

¹⁶ МАГАТЭ — Международное агентство по атомной энергии (IAEA). Публикует технические серии по ядерной безопасности, носящие рекомендательный характер для стран-членов.

¹⁷ АСУ ТП — автоматизированная система управления технологическим процессом. Программно-техническая система, обеспечивающая контроль и управление производственными процессами в реальном времени.

3.2. Мобилизация рабочих мест в атомной промышленности

Несмотря на строгие ограничения, атомная отрасль последовательно движется к цифровизации полевых рабочих мест. Традиционные бумажные процессы уступают место мобильным сценариям, позволяющим сократить трудозатраты и повысить оперативность реагирования на инциденты.

Ключевые задачи, решаемые с помощью мобильных устройств на объектах атомной промышленности:

Цифровые обходы оборудования. Сотрудник выполняет плановый обход с мобильным устройством: считывает QR-коды или NFC-метки на оборудовании, фиксирует показания, вносит отметки о техническом состоянии. Данные мгновенно передаются в систему технического обслуживания и ремонта (ТОиР¹⁸). По опыту энергетических предприятий, внедрение цифровых обходов сокращает время выполнения на 30% [20].

Электронные наряды-допуски. Наряд-допуск¹⁹ — обязательный разрешительный документ для работ в электроустановках и на объектах повышенной опасности. Переход от бумажных к электронным нарядам-допускам с подписью через КЭП²⁰ позволяет сократить время оформления документа с 30 до 20 минут, а объём бумажного документооборота — на 80% [20].

Интеграция с АСУ ТП. Мобильные устройства используются как терминалы для считывания данных с нижнего уровня АСУ ТП (датчики, контроллеры) и отображения оперативной технологической информации оперативному персоналу вне диспетчерского пункта. Архитектурно взаимодействие реализуется через защищённые протоколы (OPC-UA²¹, MQTT over TLS).

Геолокация и диспетчеризация персонала. Отслеживание местонахождения специалистов на карте объекта позволяет направить ближайшую бригаду при нештатной ситуации, минимизируя время реакции.

3.3. Применение ОС «Аврора» в структурах Росатома

Реальная практика применения сертифицированных мобильных решений в атомной отрасли наиболее полно представлена опытом Госкорпорации «Росатом».

¹⁸ТОиР — техническое обслуживание и ремонт. Информационные системы класса EAM (Enterprise Asset Management) для управления жизненным циклом промышленного оборудования.

¹⁹Наряд-допуск — разрешительный документ на производство работ повышенной опасности. Регламентируется Правилами по охране труда и отраслевыми нормативными актами.

²⁰КЭП — квалифицированная электронная подпись. Обеспечивает юридическую силу электронного документа в соответствии с Федеральным законом № 63-ФЗ «Об электронной подписи».

²¹OPC-UA (Open Platform Communications Unified Architecture) — стандарт обмена данными в промышленной автоматизации, поддерживающий шифрование и аутентификацию на уровне протокола.

В 2023 году «Росатом» принял решение о переходе на отечественные мобильные платформы: к концу 2023 года все предприятия госкорпорации отказались от корпоративных мобильных приложений на устройствах с iOS, а новые приложения стали разрабатываться под Android и «Аврору». Параллельно в ряде подразделений проводилось тестирование «доверенных» смартфонов на ОС «Аврора».

ОС «Аврора» обладает необходимым регуляторным статусом для применения на объектах КИИ первой категории: сертификат ФСТЭК (УД4, профиль А4) и сертификат ФСБ (АК3/КС3) совместно позволяют использовать систему в государственных информационных системах первого класса защищённости и на объектах КИИ [21]. Критически важной является и возможность интеграции с отечественными СКЗИ²²: «КриптоПро» реализована как встроенный компонент в сертифицированных сборках «Авроры».

Среди других крупных российских компаний, внедривших или тестирующих «Аврору», фигурируют РЖД (устройства для проводников), «Интер РАО», «Россети», «Почта России». Совокупное число устройств на «Авроре» в корпоративном сегменте к 2024 году превысило 500 тысяч единиц.

²²СКЗИ — средство криптографической защиты информации. Сертифицированные СКЗИ (КриптоПро, ViPNet) обязательны для обеспечения защищённого канала связи на объектах КИИ.

3.4. Требования к безопасности данных мобильных устройств

Применение мобильных устройств на ядерных объектах требует реализации многоуровневой модели защиты, принципиально более строгой, чем в типовой корпоративной среде.

Таблица 3.1 — Требования к защите данных на мобильных устройствах в атомной отрасли

Уровень защиты	Требование	Реализация в ОС «Аврора»
Шифрование хранилища	FDE с ГОСТ 28147-89 / ГОСТ Р 34.12 ²³	Встроено, сертифицировано ФСБ
Криптозащита канала	ГОСТ TLS (ГОСТ Р 34.10, 34.11)	КриптоПро в составе ОС
Аутентификация	Двухфакторная; КЭП для ЭДО	Поддержка токенов и сертификатов
Целостность ОС	Verified Boot, контроль подписи пакетов	Встроен в загрузчик и пакетный менеджер
Ограничение ПО	Только доверенные приложения от ОМПИ	Белый список; запрет сторонних APK
Удалённое управление	Принудительная блокировка / wipe	«Аврора Центр» (EMM)
Физическая защита	Запрет камеры, микрофона, Bluetooth на режимных зонах	Политики MDM через «Аврора Центр»
Журналирование	Аудит всех действий пользователя	Системный журнал, интеграция с SIEM

Отдельного внимания заслуживает вопрос **сетевой изоляции**. Промышленные сети АЭС (уровень АСУ ТП) физически изолированы от корпоративных и внешних сетей (принцип «воздушного зазора», air gap). Мобильное устройство, интегрируемое с нижним уровнем АСУ ТП, либо работает в полностью изолированном сегменте без выхода в интернет, либо взаимодействует через сертифицированный шлюз безопасности. Передача данных «наружу» осуществляется исключительно через однонаправленные шлюзы (data diodes), что исключает возможность обратного воздействия на технологическую сеть.

²³Источник: https://ru.wikipedia.org/wiki/%D0%93%D0%9E%D0%A1%D0%A2_34.12-2018

МАГАТЭ в технической серии NSS 33-Т фиксирует, что мобильные устройства представляют один из наиболее актуальных векторов угроз для компьютерной безопасности ядерных объектов — в первую очередь из-за рисков несанкционированного подключения к технологическим сетям и бесконтрольного использования беспроводных интерфейсов [19]. Применение сертифицированной ОС с принудительными MDM-политиками является необходимым, но недостаточным условием: оно должно дополняться организационными мерами (контроль физического доступа, запрет личных устройств в режимных зонах) и периодическим аудитом безопасности.

4. Виды мобильных приложений

4.1. Нативные приложения

4.1.1. Определение и принцип работы

Нативное приложение — программа, разработанная с использованием официального инструментария конкретной платформы и скомпилированная в машинный код целевой архитектуры процессора. Такое приложение напрямую вызывает системные API операционной системы без промежуточных слоёв абстракции, что обеспечивает максимальную производительность и полный доступ к возможностям устройства.

С архитектурной точки зрения нативное приложение работает непосредственно в процессе ОС, получая прямой доступ к:

- графическому стеку платформы (Metal на iOS, Vulkan/OpenGL ES на Android);
- системным сервисам (Push-уведомления, фоновые задачи, виджеты, расширения);
- аппаратным компонентам (камера, биометрия, NFC, Bluetooth LE, датчики) через платформенные API без дополнительных обёрток.

4.1.2. Инструменты разработки

4.1.2.1. Android: Kotlin и Android Studio

Официальным языком разработки под Android с 2019 года является **Kotlin** [22] — статически типизированный язык JVM, разработанный компанией JetBrains. Kotlin полностью совместим с Java и постепенно вытесняет его: по данным Google, более 95% из тысячи наиболее популярных Android-приложений содержат код на Kotlin [23].

Ключевые возможности Kotlin, критичные для мобильной разработки:

- **Coroutines** — механизм структурированного асинхронного программирования без callback-hell; основа для работы с сетью и базами данных в Android-приложениях;

- **Null safety** — система типов, исключаящая NullPointerException на уровне компилятора;

- **Extension functions** — расширение существующих классов без наследования, широко применяемое в Android SDK.

Официальная среда разработки — **Android Studio** (на базе IntelliJ IDEA от JetBrains) [24]. Включает: встроенный эмулятор, профилировщик (CPU, Memory, Network, Energy), Layout Inspector, систему сборки Gradle и инструменты для работы с базами данных Room и Jetpack-библиотеками.

Декларативный UI-фреймворк **Jetpack Compose**, стабильно выпущенный в 2021 году, стал рекомендованным подходом к разработке интерфейса, заменяя XML-разметку на Kotlin-код с реактивным управлением состоянием.

4.1.2.2. iOS: Swift и Xcode

Официальным языком разработки под iOS является **Swift** — компилируемый статически типизированный язык, представленный Apple в 2014 году [25]. Swift компилируется LLVM в нативный машинный код ARM64 (или x86_64 для симулятора), обеспечивая производительность, сопоставимую с C++, при высоком уровне безопасности памяти.

Ключевые особенности Swift для iOS-разработки:

- **Swift Concurrency** (`async / await`, `Actor`) — модель структурированного параллелизма, введённая в Swift 5.5; полностью интегрирована с системными API Apple;

- **Optionals** — явная работа с отсутствием значения на уровне системы типов, аналог `?`-типов в Kotlin;

- **Protocol-oriented programming** — архитектурная парадигма, предпочтительная в экосистеме Apple вместо классического ООП.

Официальная среда разработки — **Xcode** [26]. Включает: симулятор устройств, Instruments (профилировщик производительности, утечек памяти, потребления энергии), Interface Builder и инструменты для подписи и дистрибуции приложений. Xcode доступен исключительно на macOS, что является одним из ключевых ограничений iOS-разработки.

UI-фреймворк **SwiftUI** (с 2019 года) стал декларативной альтернативой UIKit, предоставляя реактивное управление состоянием через `@State`, `@Binding`, `@ObservedObject`. UIKit при этом остаётся актуальным для сложных кастомных компонентов и legacy-кодовых баз.

4.1.3. Преимущества и недостатки

Таблица 4.1 — Преимущества и недостатки нативной разработки

Преимущества	Недостатки
Максимальная производительность: нет промежуточных слоёв и рантайм-оверхеда	Две независимые кодовые базы: Android (Kotlin) и iOS (Swift)
Полный доступ ко всем системным API и аппаратным возможностям сразу после релиза платформы	Пропорционально вдвое больше ресурсов: две команды, два ревью, два CI-пайплайна
Нативный UX: платформенные компоненты, анимации и жесты соответствуют гайдлайнам (HIG, Material Design)	Разные языки и парадигмы требуют разных компетенций от разработчиков
Лучшая интеграция с платформенными функциями: виджеты, Live Activities, Shortcuts, интенды	Высокая стоимость разработки и поддержки для компаний, охватывающих обе платформы
Своевременная поддержка новых возможностей ОС в день релиза	Дублирование бизнес-логики: изменения необходимо вносить синхронно в обоих репозиториях
Инструменты профилирования и отладки максимально интегрированы с платформой	Длительный онбординг: глубокое знание платформы требует значительного времени

Нативный подход остаётся предпочтительным для приложений, где критична производительность (графические редакторы, игры, AR-приложения), требуется плотная интеграция с аппаратными компонентами (медицинские устройства, промышленные сканеры) или необходима немедленная поддержка новых возможностей платформы. В корпоративном контексте нативная разработка оправдана также для флагманских продуктов с большой аудиторией, где UX-качество критично для бизнеса.

4.2. Гибридные и кросс-платформенные приложения

4.2.1. Определение и мотивация

Кросс-платформенная разработка возникла как ответ на главный экономический недостаток нативного подхода — необходимость поддерживать две независимые кодовые базы для Android и iOS. Идея проста: если бизнес-логика

приложения одинакова на обеих платформах, нет смысла писать её дважды на разных языках.

Различные фреймворки по-разному отвечают на вопрос о том, **что именно** следует разделять:

- «**написать один раз, запустить везде**» (Write Once, Run Anywhere) — полная унификация кода, включая UI; подход Flutter и Xamarin;

- «**написать бизнес-логику один раз, UI — нативно**» — разделение слоёв: бизнес-логика и данные общие, UI — нативный на каждой платформе; подход Kotlin Multiplatform (KMP).

Эти философии определяют принципиально разные компромиссы между скоростью разработки, UX-качеством и долгосрочной поддерживаемостью кода.

4.2.2. React Native

React Native — фреймворк от компании Meta (Facebook), выпущенный в 2015 году [8]. Основан на JavaScript (TypeScript) и концепциях React: компонентная модель, декларативный UI, однонаправленный поток данных.

4.2.2.1. Архитектура

Исторически React Native использовал так называемый **Bridge** — асинхронный JSON-мост между JavaScript-поток и нативным UI-поток. Каждый вызов нативного компонента сериализовался в JSON, передавался через мост и десериализовался на нативной стороне. Этот механизм являлся основным источником проблем с производительностью в интенсивных сценариях (анимации, жесты, длинные списки).

Начиная с 2022 года React Native постепенно переходит на **Новую архитектуру** (New Architecture), включающую:

- **JSI** (JavaScript Interface) — синхронный низкоуровневый интерфейс, позволяющий JS напрямую вызывать нативный код без сериализации;

- **Fabric** — новый рендерер UI, построенный на C++ и обеспечивающий синхронный доступ к дереву компонентов;

- **TurboModules** — ленивая инициализация нативных модулей вместо загрузки всех модулей при старте приложения.

4.2.2.2. Рендеринг

React Native не рисует UI сам — он отображает **нативные компоненты** платформы. `<View>` превращается в `android.view.View` на Android и в `UIView` на iOS. Это означает, что приложение выглядит нативно и автомати-

чески следует дизайн-системе платформы, однако ограничивает возможности кастомизации и создаёт риск расхождения поведения между платформами.

4.2.2.3. Экосистема

JavaScript/TypeScript — наиболее распространённый стек в индустрии, что даёт React Native огромное преимущество в доступности кадров. Экосистема npm содержит сотни готовых библиотек для RN. Крупные компании, использующие React Native в production: Meta, Microsoft, Shopify, Wix.

4.2.3. Flutter

Flutter — UI-фреймворк от Google, представленный в 2018 году [7]. Язык разработки — **Dart**, компилируемый AOT (Ahead-of-Time) в нативный машинный код ARM64/x86_64.

4.2.3.1. Архитектура и рендеринг

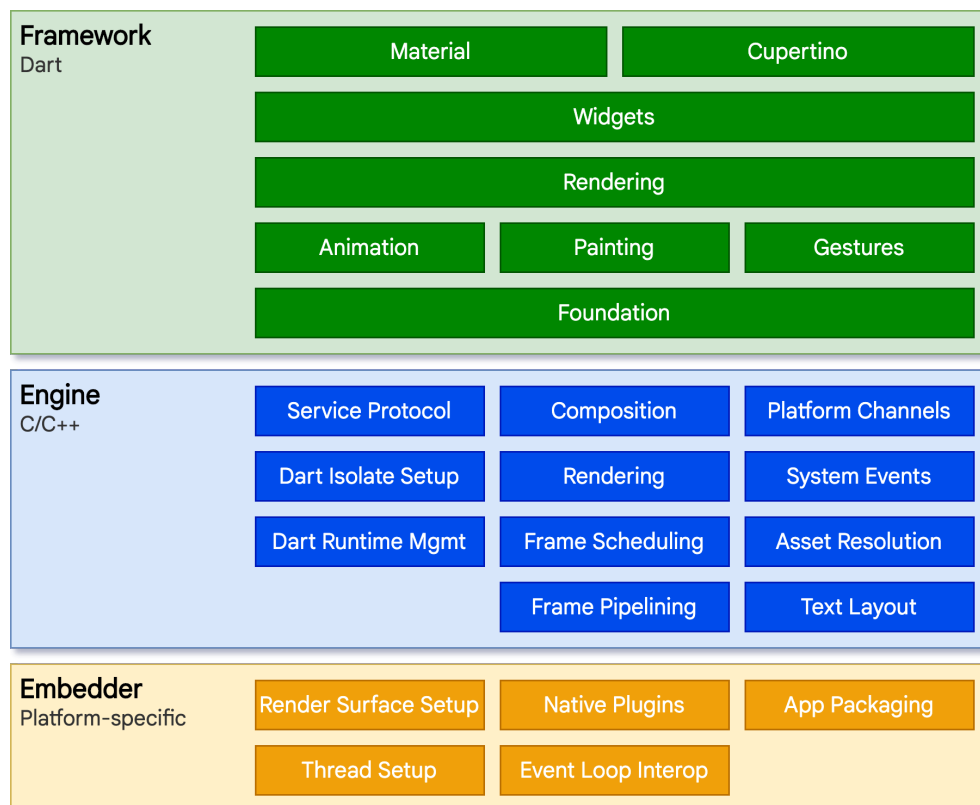


Рисунок 4.1 — Архитектура фреймворка Flutter [7]

Ключевое архитектурное решение Flutter радикально отличается от React Native: Flutter **не использует нативные UI-компоненты**. Вместо этого он получает чистый холст от платформы (через OpenGL ES / Metal / Vulkan) и самостоятельно рисует на нём **каждый пиксель** с помощью собственного движка Skia (Flutter 2) / Impeller (Flutter 3+) (см. Рис. 4.1).

Это даёт принципиальные преимущества:

- **Pixel-perfect консистентность** между платформами: один и тот же виджет выглядит одинаково на Android, iOS, Web и Desktop;
- **Независимость от платформенного UI-стека**: обновления Android или iOS не ломают визуальную составляющую приложения;
- **Стабильные 60/120 fps** даже на сложных анимациях, поскольку рендеринг выполняется нативным кодом на GPU.

Движок Flutter написан на C++ и выполняется в отдельном потоке. Dart-код компилируется AOT, что исключает JIT-паузы в production.

4.2.3.2. Архитектура слоёв Flutter

Стек Flutter состоит из трёх уровней:

1. **Framework** (Dart) — виджеты, анимации, жесты, навигация, Material / Cupertino дизайн-системы;
2. **Engine** (C++) — Skia/Impeller, Dart VM, Platform Channels, текстовый движок;
3. **Embedder** (платформено-специфичный) — точка входа для каждой целевой платформы (Android Activity / iOS UIViewController).

4.2.3.3. Состояние экосистемы

pub.dev содержит более 40 000 пакетов для Flutter. Фреймворк поддерживает шесть платформ из одной кодовой базы: Android, iOS, Web, macOS, Windows, Linux — что делает его уникальным с точки зрения охвата.

4.2.4. Kotlin Multiplatform (KMP) и Compose Multiplatform (CMP)

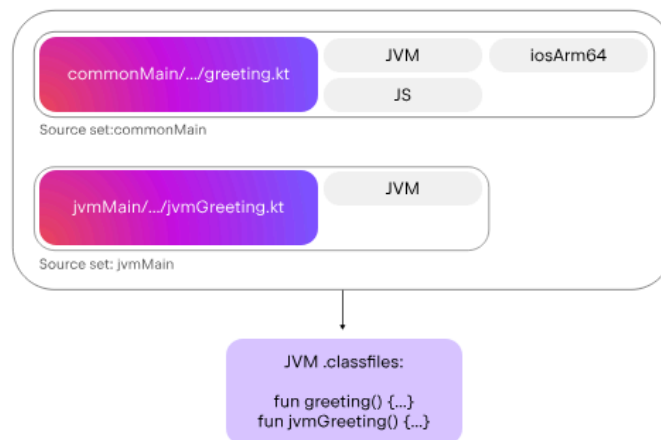


Рисунок 4.2 — Пример архитектуры KMP приложения[22]

Kotlin Multiplatform (KMP) — технология JetBrains, получившая статус Stable в ноябре 2023 года [27]. В отличие от Flutter и React Native, KMP реализует принципиально иной подход к кросс-платформенности.

4.2.4.1. Философия KMP: общий код, нативный UI

KMP не стремится унифицировать UI. Его цель — вынести в общий модуль слои, где унификация действительно оправдана:

- **Domain-слой**: бизнес-логика, use cases, модели данных;
- **Data-слой**: сетевые запросы (Ktor), локальная БД (Room3/Realm/SQLDelight), репозитории, маппинг DTO;
- **Presentation-логика**: ViewModel, StateFlow, обработка событий — при использовании паттерна MVI/MVVM.

UI при этом пишется нативно: **Jetpack Compose** на Android и **SwiftUI** на iOS. Это означает, что iOS-разработчик пишет SwiftUI как обычно — только бизнес-логику он получает из общего KMP-модуля, а не реализует её заново.

4.2.4.2. Compose Multiplatform (CMP)

Compose Multiplatform — надстройка над KMP от JetBrains, распространяющая декларативный UI-фреймворк Jetpack Compose на несколько платформ [28]:

- **Android**: стабильно, это нативный Jetpack Compose;
- **Desktop (JVM)** (macOS, Windows, Linux): стабильно с 2023 года;
- **iOS**: Beta с 2024 года (Compose Multiplatform 1.6+);
- **Web** (Wasm): Alpha.

CMP позволяет использовать единый Compose-UI на всех платформах, сохраняя при этом возможность встраивать платформу-специфичные компоненты через `expect / actual` механизм.

Механизм `expect / actual` — ключевой элемент KMP/CMP. Разработчик объявляет `expect fun getPlatformName(): String` в общем модуле, а в каждом платформенном модуле предоставляет `actual`-реализацию. Компилятор гарантирует, что реализация есть на каждой целевой платформе.

4.2.4.3. Позиционирование KMP на рынке

KMP активно принимают крупные компании: Netflix использует его для разделения логики рекомендательных алгоритмов, Cash App (Block) — для финансовых вычислений, Philips — в медицинских приложениях [27]. В российском контексте KMP/CMP выгоден тем, что Android-стек (Kotlin) уже является де-факто стандартом, а переход к KMP не требует смены языка.

4.2.5. Xamarin и .NET MAUI

Xamarin — фреймворк Microsoft на базе .NET и C#, позволявший писать кросс-платформенные мобильные приложения с 2011 года. В 2023 году Xamarin официально устарел (End of Life — май 2024) и заменён фреймворком **.NET MAUI** (Multi-platform App UI) [29].

MAUI расширяет концепцию Xamarin.Forms: один проект на C# компилируется для Android, iOS, macOS и Windows. MAUI использует нативные UI-контролы каждой платформы (аналогично React Native), а не кастомный рендерер (как Flutter). Основная аудитория — корпоративные команды с экспертизой в .NET и существующими C#-бэкендами.

4.2.6. Сравнение кросс-платформенных подходов

Таблица 4.2 — Сравнение кросс-платформенных фреймворков

Критерий	React Native	Flutter	KMP / CMP	.NET MAUI
Язык	JS / TypeScript	Dart	Kotlin	C#
Рендеринг UI	Нативные компоненты	Собственный движок (Impeller)	Нативный UI / Compose (CMP)	Нативные компоненты
Shared-код	UI + логика	UI + логика	Только логика (UI опционально)	UI + логика
Производительность	Хорошая (New Arch.)	Отличная	Нативная	Хорошая
Зрелость	Высокая (с 2015)	Высокая (с 2018)	Stable с 2023 (CMP iOS — Beta)	Средняя (с 2022)
Доступ к нативным API	Platform Channels	Platform Channels	Прямой (expect/actual)	Handlers
Целевые платформы	Android, iOS, Web, Desktop	Android, iOS, Web, macOS, Win, Linux	Android, iOS, Desktop, Server	Android, iOS, macOS, Windows
Экосистема (пакеты)	npm (~3 000 RN-пакетов)	pub.dev (~40 000)	Maven / CocoaPods	NuGet
Типичная аудитория	Web-команды с React-экспертизой	Мобильные команды, стартапы	Android-команды, enterprise	.NET / Microsoft-стек

4.2.7. Преимущества и компромиссы кросс-платформенного подхода

Общий выигрыш кросс-платформенного подхода по сравнению с нативной разработкой — экономия 30–50% ресурсов на разработку и поддержку при охвате двух платформ. Однако этот выигрыш не бесплатен.

Компромисс 1: доступ к платформенным API. Новые системные возможности (Live Activities, Dynamic Island на iOS; Predictive Back на Android) появляются сначала в нативных SDK. Кросс-платформенные фреймворки под-

держивают их с задержкой — от нескольких недель (Flutter/RN) до мгновенно (KMP, поскольку UI нативный).

Компромисс 2: размер приложения. Flutter включает собственный движок (~4–7 МБ), React Native — JS-рантайм. Нативные приложения, как правило, компактнее.

Компромисс 3: отладка. Стек ошибок проходит через промежуточные слои фреймворка, что усложняет диагностику низкоуровневых проблем по сравнению с нативной разработкой.

Компромисс 4: нишевые платформы. Для «Авроры» (Qt/QML) ни один из перечисленных фреймворков не поддерживается официально. KMP теоретически применим на уровне бизнес-логики (Kotlin с помощью `QtBinding` собирается в нативную статическую/динамическую C++ библиотеку), однако UI для «Авроры» необходимо писать на Qt/QML отдельно.

4.3. Прогрессивные веб-приложения (PWA)

4.3.1. Определение и концепция

Progressive Web Application (PWA) — веб-приложение, использующее современные браузерные API для достижения возможностей, традиционно доступных только нативным приложениям: работа офлайн, push-уведомления, установка на домашний экран, доступ к аппаратным компонентам устройства.

Термин введён инженером Google Алексом Расселом в 2015 году. Концептуальная основа PWA — **прогрессивное улучшение** (progressive enhancement): приложение работает в любом браузере как обычный сайт, но в современных браузерах с поддержкой соответствующих API «прокачивается» до почти нативного опыта [30].

PWA не является отдельным фреймворком или технологией — это набор критериев и паттернов, которым может соответствовать любое веб-приложение. Google определяет три ключевых критерия: **capable** (использует современные веб-API), **reliable** (работает офлайн или в условиях нестабильной сети), **installable** (может быть установлено на устройство).

4.3.2. Ключевые технологии

4.3.2.1. Service Worker

Service Worker — JavaScript-скрипт, выполняющийся в отдельном потоке браузера, независимо от основной страницы [31]. Это ключевой компонент PWA: Service Worker перехватывает сетевые запросы приложения и может:

- возвращать ответы из кэша при отсутствии сети;
- реализовывать стратегии кэширования (Cache First, Network First, Stale-While-Revalidate²⁴);
- выполнять фоновую синхронизацию данных (Background Sync API);
- принимать и отображать push-уведомления, даже когда приложение закрыто.

Service Worker работает исключительно по HTTPS (или на localhost), что является базовым требованием безопасности.

4.3.2.2. Web App Manifest

Web App Manifest — JSON-файл, описывающий метаданные приложения для браузера [32]:

```
1 {
2   "name": "Корпоративный портал",
3   "short_name": "Портал",
4   "start_url": "/",
5   "display": "standalone",
6   "background_color": "#ffffff",
7   "theme_color": "#1a73e8",
8   "icons": [
9     { "src": "/icon-192.png", "sizes": "192x192", "type":
"image/png" },
10    { "src": "/icon-512.png", "sizes": "512x512", "type":
"image/png" }
11  ]
12 }
```

Параметр `display: standalone` убирает браузерный chrome (адресную строку, кнопки навигации) — приложение выглядит как нативное. Браузер использует манифест для формирования иконки на рабочем столе, сплеш-экрана при запуске и поведения при добавлении на домашний экран.

4.3.2.3. Push API и Notifications API

Push API позволяет серверу отправлять сообщения в Service Worker даже при закрытом приложении [33]. Технически это реализуется через протокол

²⁴Stale-While-Revalidate — стратегия кэширования: браузер немедленно отдаёт устаревшую кэшированную версию, одновременно обновляя её в фоне. Оптимальна для данных, где небольшая задержка актуализации допустима.

²⁵VAPID (Voluntary Application Server Identification) — стандарт аутентификации push-сервера. Исключает возможность отправки push-уведомлений от неавторизованных серверов.

Web Push (RFC 8030) с шифрованием VAPID²⁵. Service Worker получает push-событие и вызывает Notifications API для отображения уведомления в ОС.

4.3.2.4. Дополнительные современные API

Современные браузеры предоставляют PWA доступ к аппаратным ресурсам, которые ещё недавно были привилегией нативных приложений:

- **Web Bluetooth** — взаимодействие с BLE-устройствами;
- **Web NFC** — чтение/запись NFC-меток (только Android Chrome);
- **File System Access API** — работа с локальной файловой системой;
- **WebAuthn** — аутентификация через биометрию устройства (Touch ID, Face ID, Windows Hello);
- **Background Sync** — отложенная отправка данных при восстановлении сети;
- **IndexedDB** — структурированное локальное хранилище для офлайн-данных.

4.3.3. Ограничения на iOS и Android

Поддержка PWA в iOS исторически значительно отстаёт от Android, что обусловлено стратегическими интересами Apple по защите App Store.

Таблица 4.3 — Поддержка ключевых PWA-возможностей на iOS и Android

Возможность	Android Chrome	iOS Safari
Service Worker	Полная	С iOS 11.3
Web App Manifest	Полная	Частичная
Push-уведомления	Полная	С iOS 16.4 (только Home Screen)
Background Sync	есть	нет
Web Bluetooth	есть	нет
Web NFC	есть	нет
File System Access API	есть	нет
Офлайн-режим	Полная	Частичная (лимит кэша 50 МБ)
Установка на экран	Нативный диалог	Ручное добавление через «Поделиться»
Badge API (счётчик иконки)	есть	С iOS 16.4

Ключевые ограничения iOS по состоянию на 2025 год:

Push-уведомления появились на iOS только в версии 16.4 (март 2023 года) и работают исключительно для приложений, добавленных на домашний экран — при открытии в браузере Safari push-уведомления по-прежнему недоступны.

Хранилище. Safari агрессивно очищает кэш Service Worker при длительном неиспользовании приложения (порог — около 7 дней). На Android Chrome данные PWA хранятся постоянно, пока пользователь явно не удалит их.

Отсутствие промпта установки. Android Chrome показывает нативный диалог «Добавить на главный экран» при соответствии Manifest-критериям. iOS требует от пользователя вручную найти пункт «На экран «Домой»» в меню «Поделиться» Safari — что существенно снижает конверсию установки.

В марте 2024 года под давлением Digital Markets Act в ЕС Apple открыла возможность установки сторонних браузеров с собственными движками. Это потенциально позволит Chrome и Firefox на iOS поддерживать полный спектр

PWA API, однако практическое распространение этой возможности остаётся ограниченным.

4.3.4. Сценарии применения и ограничения в корпоративной среде

PWA обоснован в ситуациях, когда важны три фактора: широкая доступность (URL вместо установки из магазина), быстрое обновление (без прохождения модерации App Store / Google Play) и единая кодовая база для мобильного и десктопного веб.

Типичные корпоративные сценарии:

— **Внутренние порталы и дашборды** — HR-системы, корпоративные справочники, системы заявок; пользователи работают преимущественно онлайн, офлайн-требования минимальны;

— **Field service с базовым офлайн-режимом** — формы заявок, чеклисты, простые справочники; данные синхронизируются при появлении сети через Background Sync;

— **Лёгкие клиенты ERP/CRM** — доступ к корпоративным данным без установки тяжёлых нативных приложений на устройства сотрудников (актуально при BYOD-модели).

Однако PWA имеет существенные ограничения, критичные именно для корпоративного контекста:

— **MDM-управление.** PWA не являются «приложениями» с точки зрения MDM-систем. Их нельзя принудительно установить через Android Enterprise / Apple MDM Protocol, управлять их разрешениями или гарантировать версию. Корпоративные MAM²⁶-политики к PWA неприменимы;

— **Безопасность данных.** PWA хранит данные в браузерном хранилище (IndexedDB, Cache API), которое не шифруется средствами ОС. Это неприемлемо для данных с ограниченным доступом. Нативные приложения могут использовать Android Keystore / iOS Secure Enclave;

— **Аппаратные ограничения iOS.** Отсутствие Web Bluetooth, Web NFC и File System Access API на iOS делает PWA непригодным для промышленных сценариев с периферийным оборудованием (сканеры, считыватели меток);

— **Сертифицированные среды.** Для объектов КИИ и государственных ИС требуется использование сертифицированного ПО. Браузер, исполняющий

²⁶MAM (Mobile Application Management) — управление мобильными приложениями как подмножество EMM. Позволяет управлять политиками конкретных приложений: шифрование данных приложения, ограничения copy-paste, удалённое уничтожение данных.

PWA, должен быть сертифицирован ФСТЭК — что на практике исключает большинство PWA-сценариев в чувствительных секторах.

Таким образом, PWA занимает нишу между мобильным сайтом и полноценным нативным приложением: он уместен для внутренних инструментов с невысокими требованиями к безопасности и аппаратной интеграции, но не является полноценной заменой нативному или кросс-платформенному приложению в большинстве enterprise-сценариев.

5. Стек технологий мобильной разработки

Выбор стека технологий определяет не только скорость разработки, но и долгосрочную поддерживаемость продукта, доступность кадров и возможности интеграции с корпоративной инфраструктурой. В данной главе рассматриваются языки, фреймворки, инструменты CI/CD²⁷, тестирования и безопасности, составляющие современный стек мобильной разработки.

5.1. Языки и платформенные фреймворки

5.1.1. Kotlin

Kotlin — основной язык разработки под Android и ключевой язык KMP-экосистемы. Компилятор Kotlin (kotlinc) поддерживает несколько бэкендов [22]:

- **Kotlin/JVM** — компиляция в JVM-байткод; стандарт для Android- и Десктоп-приложений и серверного Kotlin;

- **Kotlin/Native** — компиляция через LLVM в нативный машинный код; используется для iOS (arm64), Linux (x64, arm64), macOS, Windows; является основой для KMP на платформах без JVM;

- **Kotlin/JS** — компиляция в JavaScript; применяется для веб-таргетов в KMP-проектах;

- **Kotlin/Wasm** — компиляция в WebAssembly; экспериментальный таргет для Compose Multiplatform Web.

Для мобильной разработки ключевыми являются механизмы языка: **Coroutines** + **Flow** для реактивного асинхронного программирования, **Serialization** (`kotlinx.serialization`) для работы с JSON/Protobuf без рефлексии, **Sealed classes** для моделирования состояний UI.

5.1.2. Swift

Swift используется исключительно для нативной iOS-разработки и серверных приложений (Vapor, Hummingbird). В контексте KMP Swift взаимодействует с Kotlin через генерируемые заголовки Objective-C: KMP-модуль компилируется в нативный фреймворк `.xcframework`, который подключается в Xcode-проект как обычная зависимость. Взаимодействие прозрачно для Swift-разработчика: он вызывает Kotlin-классы как обычные Swift-типы.

²⁷CI/CD — Continuous Integration / Continuous Delivery. Практика автоматизации сборки, тестирования и доставки программного обеспечения. Позволяет выпускать обновления приложений быстро и с предсказуемым качеством.

5.1.3. Dart

Dart — язык Google, разработанный специально под Flutter [7]. Компилируется AOT в нативный код для production и JIT для режима разработки (что обеспечивает Hot Reload — обновление UI без перезапуска приложения). Dart не является кросс-платформенным языком вне Flutter: экосистема pub.dev замкнута на Flutter/Dart-приложения.

5.1.4. JavaScript / TypeScript

TypeScript — типизированное надмножество JavaScript — используется в React Native и для разработки PWA. Статическая типизация TypeScript решает основные проблемы надёжности JavaScript-кода, однако не устраняет накладных расходов JS-рантайма при выполнении.

5.1.5. C++ и Qt/QML

Для разработки приложений под ОС «Аврора» основным инструментарием является Qt-фреймворк и декларативный язык QML. C++ обеспечивает бизнес-логику и интеграцию с системными библиотеками, QML — описание UI. Это принципиально отличает «Аврору» от Android/iOS-платформ и создаёт отдельную нишу компетенций, не пересекающуюся с основными мобильными стеками.

5.2. Кросс-платформенные фреймворки: детальное сравнение

5.2.1. Flutter vs. React Native vs. KMP/CMP

Таблица 5.1 — Детальное сравнение Flutter, React Native и KMP/CMP

Критерий	Flutter	React Native	KMP / CMP
Стартовое время сборки	Среднее (AOT)	Быстрое (JS bundler)	Среднее (Gradle + LLVM)
Hot Reload	Да (< 1 сек)	Да (Fast Refresh)	Частично (Android)
Размер приложения	+4–7 МБ (движок)	+3–5 МБ (JS-рантайм)	Минимальный оверхед
Поддержка «Авроры»	Нет	Нет	Да (Alpha) — см. ниже
Типобезопасность	Dart (строгая)	TypeScript (опциональная)	Kotlin (строгая)
Доступ к нативным API	Platform Channels (async)	JSI (sync, New Arch.)	Прямой (C-interop)
Поддержка Coroutines	Нет (собственный async)	Нет (Promise/async-await)	Да (native)
Внедрение в enterprise (РФ)	Активное	Умеренное	Растущее

5.3. KMP и ОС «Аврора»: Aurora Multiplatform

	Веха	Дата	Что это значит
KMP	QtBindings Gradle Plugin	2 квартал 2025	Можно собрать и запустить kmm-production-sample путём генерации Qt-обёрток и написания нативного UI для приложения с помощью QML.
	Aurora OS KMP Gradle Plugin	3 квартал 2025	Начинают появляться в доступе портированные для ОС Aurora KMP библиотеки, появляются цели сборки для ОС Аврора.
	Aurora OS репозиторий Maven	3 квартал 2025	Все портированные и созданные артефакты могут быть подключены в качестве зависимостей проекта из Maven репозитория ОС Аврора
	Compose Multiplatform	1 квартал 2026	Возможность писать общий UI приложения для ОС Аврора
	Переход из R&D в продакшен	2026	Появится возможность создавать продуктовые приложения на KMP + CMP для ОС Аврора

Рисунок 5.1 — Этапы развития Aurora Multiplatform [34]

Особого внимания заслуживает инициатива ОМП по поддержке KMP для ОС «Аврора» — проект **Aurora Multiplatform**, находящийся в стадии Alpha (версия 0.0.3) (см. Рис. 5.1). Это стратегически важное направление: оно позволяет разработчикам, пишущим KMP-приложения под Android и iOS, повторно использовать общий Kotlin-код для «Авроры», дописав лишь нативный UI на Qt/QML.

5.3.1. Архитектурная основа

ОС «Аврора», как и iOS, не имеет JVM. По этой причине KMP доступен для «Авроры» через нативные таргеты Kotlin/Native: `linuxX64` (эмулятор) и `linuxArm64` (физическое устройство) [35]. Kotlin-код компилируется через LLVM в нативную статическую или динамическую C-библиотеку, которая затем подключается к Qt/QML-приложению.

5.3.2. Компоненты Aurora Multiplatform

Проект включает четыре ключевых компонента [34]:

1. Aurora KInterop — набор KMP-библиотек для взаимодействия с системными API ОС «Аврора» [36]. Библиотеки используют механизм **Kotlin C-interop** (kotlinlang.org/docs/native-c-interop.html) для вызова низкоуровневых C-библиотек платформы из Kotlin-кода. Каждая библиотека самодостаточна и может использоваться независимо. Публикуется через локальный Maven-репозиторий командой `./gradlew publishToMavenLocal`.

2. QtBindings — плагин-кодогенератор, решающий фундаментальную проблему интеграции KMP с Qt [35].

При «голом» использовании Kotlin/Native возникает ряд трудностей: генерируемый заголовочный C-файл содержит тысячи строк всей бизнес-логики проекта; экспортируемый интерфейс не поддерживает стандартные коллекции (List, Map); ресурсы необходимо освобождать вручную через `DisposeString` и `DisposeStablePointer`; Kotlin Coroutines не экспортируются в C-интерфейс; отсутствует объектно-ориентированный интерфейс классов (в каждый метод передаётся указатель на объект).

QtBindings автоматизирует решение всех этих проблем:

— генерирует Qt C++-обёртки для верхнеуровневых Kotlin-классов и функций; — обеспечивает поддержку Coroutines через `QFuture` (стандартный Qt-механизм асинхронности); — автоматически конвертирует Kotlin `List` / `MutableList` в `QList`; — автоматически управляет временем жизни объектов, устраняя необходимость ручного освобождения ресурсов.

3. Compose Multiplatform для «Авроры» — адаптация CMP для Linux-таргета, на котором основана ОС «Аврора» [37]. CMP изначально не рассчитан на нативное Linux-окружение, поэтому ОМП ведёт работу по адаптации ключевых библиотек под Linux-таргет сборки. Библиотеки КМР с доступной целевой сборкой Linux в целом совместимы с «Авророй», однако часть CMP-библиотек требует доработки — ОМП формирует и поддерживает актуальный список поддерживаемых пакетов.

4. BuildTools — Gradle-плагин для управления сборочной инфраструктурой [38]:

— создание `sysroot`²⁸-директории (поддерживаются macOS, Linux, Windows); — сборка RPM²⁹-пакета с приложением; — запуск приложения на эмуляторе и физическом устройстве.

²⁸Sysroot — корневая файловая система целевой платформы, используемая при кросс-компиляции. Содержит заголовочные файлы и библиотеки целевой ОС, необходимые компилятору на машине разработчика.

²⁹RPM (Red Hat Package Manager) — формат пакетов, используемый в дистрибутивах на базе RPM (RHEL, Fedora, ROSA). ОС «Аврора» использует RPM в качестве основного формата пакетов приложений.

5.3.3. Практическая схема КМР-проекта под «Аврору»

Типовой КМР-проект с поддержкой «Авроры» имеет следующую структуру модулей:

```
1 project/
2   └─ shared/                # Общий КМР-модуль
3       └─ commonMain/        # Бизнес-логика (Domain + Data)
4           └─ androidMain/   # Android-специфика
5               └─ iosMain/   # iOS-специфика
6                   └─ linuxArm64Main/ # Аврора-специфика (KInterop)
7   └─ androidApp/            # Android UI (Jetpack Compose)
8   └─ iosApp/                # iOS UI (SwiftUI)
9   └─ auroraApp/             # Аврора UI (Qt/QML)
10      └─ src/
11          └─ main.qml
12          └─ KmpBridge.cpp   # Qt-обёртки, генерируемые
                               QtBindings
```

Таким образом, команда пишет бизнес-логику один раз на Kotlin в `shared/commonMain`, а UI адаптирует нативно для каждой платформы. «Аврора» при этом включается в КМР-матрицу как полноправная цель сборки наравне с Android и iOS.

5.4. Backend и API

5.4.1. REST

REST (Representational State Transfer) — архитектурный стиль взаимодействия, основанный на HTTP-протоколе. Остаётся наиболее распространённым подходом для мобильного backend благодаря простоте, широкой инструментальной поддержке и хорошей кэшируемости. Для Kotlin/Android стандартными библиотеками являются **Ktor Client** (официальная КМР-библиотека JetBrains, работающая на всех платформах КМР) и **Retrofit** (Android-специфичная, не поддерживает iOS/«Аврору»).

5.4.2. GraphQL

GraphQL — язык запросов к API, разработанный Meta в 2012 году. Ключевое преимущество перед REST — клиент запрашивает ровно те поля, которые ему нужны, исключая over-fetching³⁰ и under-fetching. Для мобильных

³⁰Over-fetching — получение избыточных данных от сервера. Типичная проблема REST: эндпоинт `/user` возвращает полный объект пользователя, тогда как мобильному клиенту нужны только имя и аватар.

приложений особенно значима подписка (Subscription) через WebSocket — механизм real-time обновлений без поллинга.

5.4.3. gRPC

gRPC — фреймворк Google для вызова удалённых процедур на базе HTTP/2 и бинарной сериализации Protobuf. Превосходит REST по производительности и объёму передаваемых данных (бинарный формат в 3–10 раз компактнее JSON). В мобильном контексте применяется для высоконагруженных сценариев: стриминг данных телеметрии, real-time синхронизация.

Таблица 5.2 — Сравнение подходов к API для мобильных приложений

Критерий	REST	GraphQL	gRPC
Протокол	HTTP/1.1–2	HTTP/1.1–2	HTTP/2
Формат данных	JSON/XML	JSON	Protobuf (бинарный)
Типизация схемы	OpenAPI (опц.)	Строгая	Строгая (.proto)
Поддержка стриминга	Ограниченная	Да (Subscription)	Полная (bidirectional)
Размер payload	Базовый	Оптимальный (no over-fetch)	Минимальный
Кэширование	Нативное (HTTP)	Требует настройки	Нет
Порог входа	Низкий	Средний	Высокий

5.5. CI/CD для мобильных приложений

Непрерывная интеграция и доставка в мобильной разработке имеет специфику по сравнению с серверным ПО: необходима подпись артефактов, взаимодействие с магазинами приложений и тестирование на реальных устройствах.

5.5.1. Инструменты сборки и автоматизации

Fastlane [39] — Ruby-инструмент для автоматизации рутинных задач iOS и Android разработки: сборка, подпись, скриншоты, загрузка в App Store Connect и Google Play Console. Включает более 250 готовых действий (lanes) и поддерживает написание собственных.

GitHub Actions — облачная CI/CD-платформа с нативной поддержкой macOS-раннеров (необходимы для сборки iOS). Поддерживает матричные сборки: один workflow запускает сборку Android и iOS параллельно.

Bitrise — CI/CD-платформа, специализированная на мобильной разработке. Предоставляет готовые step-библиотеки для Fastlane, тестирования на реальных устройствах (Device Farm) и интеграции с App Store / Google Play.

GitLab CI — широко применяется в корпоративном секторе (в том числе российском) благодаря возможности self-hosted развёртывания, что критично для проектов с требованиями к размещению исходного кода на территории РФ. ОМП размещает Aurora Multiplatform на mos.hub — российской альтернативе GitLab.

5.5.2. Типовой CI/CD-пайплайн

Типовой пайплайн мобильного приложения включает следующие стадии:

1. **Lint + Static Analysis** — проверка стиля кода (ktlint, detekt для Kotlin; SwiftLint для Swift); запускается на каждый Pull Request, блокирует merge при нарушениях;
2. **Unit Tests** — быстрые тесты бизнес-логики без зависимости от устройства; время выполнения — секунды;
3. **Build** — сборка debug и release артефактов; для release обязательна подпись (Keystore для Android, Apple Distribution Certificate для iOS);
4. **UI Tests / Instrumented Tests** — запуск на эмуляторе или реальном устройстве; наиболее долгая стадия (минуты);
5. **Distribution** — загрузка артефакта в тестовый дистрибутив (Firebase App Distribution, TestFlight) или в продуктовый магазин.

5.6. Тестирование мобильных приложений

5.6.1. Пирамида тестирования

Классическая пирамида тестирования для мобильных приложений предполагает следующее соотношение: большинство тестов — быстрые unit-тесты, меньшая часть — интеграционные, и минимальная — сквозные E2E³¹-тесты.

5.6.2. Unit-тестирование

JUnit 5 + MockK (Kotlin) и **XCTest + Quick/Nimble** (Swift) — стандартные фреймворки для unit-тестирования бизнес-логики. В КМР-проектах

³¹ E2E (End-to-End) тестирование — тестирование всего пользовательского пути от UI до backend. Медленнее и дороже unit-тестов, но проверяет реальное взаимодействие компонентов.

unit-тесты бизнес-логики в `commonMain` пишутся один раз с использованием `kotlin.test` и запускаются на всех целевых платформах.

5.6.3. UI-тестирование

Espresso (Android) — официальный фреймворк Google для тестирования UI Android-приложений. Синхронизируется с UI-потокм приложения, что исключает нестабильные тесты (flaky tests) из-за задержек.

XCTest (iOS) — официальный Apple-фреймворк для UI-тестирования. Работает «снаружи» приложения через accessibility API, что приближает тесты к реальному пользовательскому взаимодействию.

Compose Testing — API для тестирования Jetpack Compose UI: `composeTestRule.onNodeWithTag()` позволяет находить компоненты по семантическим тегам без привязки к конкретной реализации.

5.6.4. E2E-тестирование

Detox — фреймворк Wix для E2E-тестирования React Native приложений. Работает на реальных устройствах и эмуляторах, поддерживает Android и iOS.

Maestro — декларативный YAML-фреймворк для E2E-тестов, набирающий популярность благодаря простоте написания сценариев и нативной поддержке как Android, так и iOS без написания кода.

5.7. Безопасность мобильных приложений

5.7.1. Обфускация кода

ProGuard / R8 (Android) — инструменты минификации и обфускации байткода. R8 является официальным компилятором Kotlin/Java начиная с Android Gradle Plugin 3.4 и совмещает оптимизацию и обфускацию в одном проходе, что сокращает размер DEX-файлов на 10–30%.

Важно понимать, что обфускация не является средством защиты: она лишь увеличивает стоимость реверс-инжиниринга, но не исключает его. Kotlin/Native (KMP) обеспечивает значительно более высокий уровень защиты кода: нативный бинарный код существенно сложнее для анализа, чем JVM-байткод.

³²MITM (Man-in-the-Middle) — атака, при которой злоумышленник перехватывает трафик между клиентом и сервером, подставляя собственный TLS-сертификат. Certificate Pinning исключает доверие к произвольным CA и принимает только заранее известный сертификат сервера.

5.7.2. Certificate Pinning

Certificate Pinning — механизм защиты от атак типа Man-in-the-Middle³²: приложение принимает только заранее определённый сертификат (или его хэш — public key pin) сервера, игнорируя любые другие, даже подписанные доверенным СА.

В Android реализуется через `network_security_config.xml` (декларативно) или через OkHttp `CertificatePinner` (программно). В iOS — через `URLSession` с кастомным `URLSessionDelegate`. В KMP/Ktor — через модуль `ktor-client-cio` с настройкой `TLSConfig`.

5.7.3. Хранение секретов

Мобильное устройство — физически доступная злоумышленнику среда, поэтому хранение секретных данных требует особого внимания.

Таблица 5.3 — Подходы к хранению секретов на мобильном устройстве

Данные	Android	iOS
Криптографические ключи	Android Keystore System	Secure Enclave / Keychain
Токены авторизации	EncryptedSharedPreferences	Keychain Services
Сертификаты	KeyStore + сертификат X.509	Keychain (идентификаторы)
Конфигурация	BuildConfig (не для секретов!) / сервер	Info.plist (не для секретов!) / сервер
Биометрия	BiometricPrompt + Keystore	LocalAuthentication + Secure Enclave

Принципиально важно: API-ключи, секретные токены и приватные сертификаты **не должны** храниться в коде приложения или ресурсах — они подлежат обязательному извлечению при статическом анализе APK/IPA. Правильный подход — хранить секреты на сервере и выдавать приложению короткоживущие токены после аутентификации.

5.7.4. OWASP Mobile Top 10

Стандартом описания угроз безопасности мобильных приложений является **OWASP Mobile Top 10** — перечень десяти наиболее распространённых категорий уязвимостей. К числу наиболее актуальных для enterprise-приложений относятся: небезопасная аутентификация, недостаточная криптография,

небезопасное хранение данных и небезопасная сетевая коммуникация. В российском контексте дополнительным требованием для приложений, работающих с персональными данными, является соответствие Федеральному закону № 152-ФЗ «О персональных данных» в части хранения и обработки данных на серверах, физически расположенных на территории РФ.

6. Особенности Enterprise-приложений

6.1. Определение и отличие от consumer-приложений

Enterprise-приложение — мобильное программное обеспечение, разрабатываемое для решения бизнес-задач организации, а не для широкой потребительской аудитории. Граница между потребительским и корпоративным приложением проходит не по интерфейсу, а по контексту эксплуатации, требованиям к надёжности и модели распространения.

Таблица 6.1 — Ключевые различия consumer- и enterprise-приложений

Параметр	Consumer	Enterprise
Аудитория	Миллионы анонимных пользователей	Конкретные сотрудники организации
Дистрибуция	App Store / Google Play	MDM / корпоративный маркет
Обновления	Опциональные, пользователь решает	Принудительные, по политике IT
Аутентификация	Логин/пароль, OAuth	SSO, LDAP/AD, PKI, МФА
Данные	Публичные или обезличенные	Конфиденциальные, часто КТ ³³
Надёжность	Деградация допустима	SLA ³⁴ с гарантированным временем работы
Интеграции	Внешние публичные API	ERP, MES, АСУ ТП, SCADA
Срок жизни	1–3 года	5–15 лет и более

В контексте атомной отрасли enterprise-приложение несёт дополнительную нагрузку: оно является частью технологического процесса, а не вспомогательным инструментом. Сбой приложения для оформления наряда-допуска или фиксации дефекта оборудования — это не «неудобство», а потенциальный риск для производственной безопасности. Это требование принципиально меняет подход к проектированию.

³³ КТ — коммерческая тайна или сведения ограниченного доступа. В атомной отрасли часть данных может относиться к государственной тайне, что влечёт отдельные требования к ПО.

³⁴ SLA (Service Level Agreement) — соглашение об уровне обслуживания. Определяет гарантированное время доступности сервиса, допустимое время восстановления и ответственность за нарушения.

6.2. Требования: надёжность, масштабируемость, интеграция

6.2.1. Надёжность и офлайн-режим

На объектах атомной промышленности — в машинных залах, реакторных отделениях, на территориях с экранированием радиосигнала — стабильное покрытие сетью Wi-Fi или сотовой связью не гарантировано. Enterprise-приложение обязано сохранять функциональность при отсутствии сети.

Архитектурный паттерн **Offline-first** предполагает:

- локальная база данных (Room на Android, CoreData / SQLite на iOS, SQLDelight в KMP) является **основным** источником данных для UI, а не кэшем;

- сетевой слой синхронизирует локальное состояние с сервером при появлении соединения, используя очередь отложенных операций (WorkManager на Android, BackgroundTasks на iOS);

- конфликты синхронизации разрешаются по детерминированной стратегии (Last Write Wins, Server Wins или бизнес-специфичная логика слияния).

Для критически важных процессов — цифровых нарядов-допуска, записей об обходе оборудования — офлайн-режим является обязательным требованием, а не опциональной функцией.

6.2.2. Масштабируемость

Enterprise-приложение в крупной госкорпорации — это не одно приложение, а семейство продуктов: десятки бизнес-процессов, сотни тысяч пользователей, разнородный парк устройств. Масштабируемость рассматривается на двух уровнях.

Масштабируемость кодовой базы. Монолитная архитектура приложения неизбежно деградирует при росте команды и числа фич. Решение — модуляризация: приложение разбивается на независимые Gradle/Swift Package-модули по функциональному или слоевому принципу. В Росатоме, где над мобильными решениями могут работать несколько команд одновременно (например, подрядчики разных предприятий ГК), модульность позволяет командам работать изолированно и независимо публиковать обновления своих модулей.

Масштабируемость инфраструктуры. Backend мобильного enterprise-приложения должен выдерживать пиковые нагрузки: утренняя смена на АЭС — это тысячи одновременных сессий, открывающихся в течение нескольких минут. Горизонтальное масштабирование backend-сервисов, CDN для статики,

балансировка нагрузки и graceful degradation³⁵ — обязательные требования к архитектуре.

6.2.3. Интеграция с корпоративными системами

Мобильное enterprise-приложение редко существует изолированно — оно является точкой доступа к данным существующих корпоративных систем.

Таблица 6.2 — Типичные интеграции мобильного enterprise-приложения в атомной отрасли

Система	Назначение	Протокол интеграции
ERP (SAP, 1C)	Управление ресурсами, закупки, финансы	REST API / OData / RFC
MES ³⁶	Управление производственными заданиями, ТОиР	REST / SOAP
АСУ ТП / SCADA ³⁷	Телеметрия, параметры оборудования	OPC-UA / MQTT over TLS
ЭДО ³⁸	Электронные документы, согласования	REST / SOAP + КЭП
LDAP / Active Directory	Аутентификация, справочник сотрудников	LDAP / Kerberos / SAML
PKI-инфраструктура	КЭП для ЭДО и критических операций	PKCS#11 / GOST TLS
GIS / ГЛОНАСС	Геолокация на территории объекта	NMEA / REST

Принципиально важен выбор протокола интеграции с АСУ ТП: прямой доступ мобильного устройства к технологической сети недопустим по соображениям безопасности. Интеграция реализуется через изолированный API-

³⁵Graceful degradation — «мягкая деградация». Способность системы сохранять частичную функциональность при отказе отдельных компонентов. Противоположность «жесткого» отказа всей системы.

³⁶MES (Manufacturing Execution System) — система управления производственными процессами. Обеспечивает оперативный контроль производства между уровнем ERP и АСУ ТП.

³⁷SCADA (Supervisory Control and Data Acquisition) — система диспетчерского управления и сбора данных. Верхний уровень автоматизации: визуализация состояния оборудования, управление технологическим процессом.

³⁸ЭДО — электронный документооборот. Системы: Directum, Tessa, DocsVision — распространены в госкорпорациях.

шлюз, физически расположенный в DMZ³⁹, с однонаправленной передачей данных из технологической сети в мобильное приложение.

6.3. Архитектурные паттерны

6.3.1. Clean Architecture

Clean Architecture — архитектурный подход, основанный на строгом разделении ответственности через концентрические слои зависимостей. Внутренние слои не зависят от внешних; зависимости направлены строго внутрь [40].

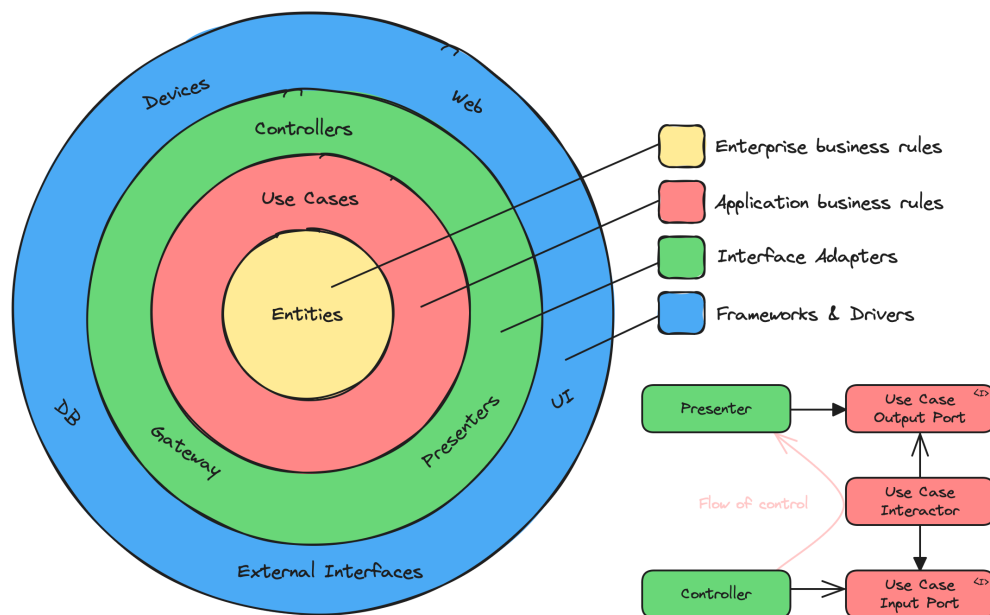


Рисунок 6.1 — Clean Architecture по Роберту Мартину [40]

В мобильном приложении классическая реализация (см. Рис. 6.1):

1. **Domain-слой** (ядро) — бизнес-сущности (**Entity**), бизнес-правила (**UseCase**), интерфейсы репозитория. Чистый Kotlin/Swift без зависимостей от Android/iOS SDK. В KMP-проекте — `commonMain`;
2. **Data-слой** — реализации репозитория, источники данных (сетевой клиент, локальная БД, кэш), маппинг DTO → Domain Entity. Зависит от фреймворков (Retrofit, Room, Ktor, SQLDelight);
3. **Presentation-слой** — ViewModel, State, события UI. Зависит от Domain, но не от Data. Тестируется unit-тестами без UI;
4. **UI-слой** — Composable-функции / SwiftUI-вью / QML. Зависит только от Presentation. Содержит минимум логики.

³⁹DMZ (Demilitarized Zone) — демилитаризованная зона. Сегмент сети, изолированный от корпоративной и технологической сетей межсетевыми экранами. Используется для размещения сервисов, доступных из нескольких сегментов.

Clean Architecture в атомной отрасли особенно ценна долгосрочной перспективой: приложение, написанное в 2024 году, может эксплуатироваться до 2035 года. За это время сменяются UI-фреймворки, обновятся версии Android, возможно — изменится целевая платформа (например, переход на «Аврору»). Если Domain-слой изолирован от платформенных зависимостей, такая миграция ограничивается переписыванием UI и Data-слоёв без затрагивания бизнес-логики.

6.3.2. MVVM и MVI

MVVM (Model-View-ViewModel) — наиболее распространённый паттерн в Android (Jetpack ViewModel) и iOS (Combine + ObservableObject). ViewModel хранит состояние UI, реагирует на пользовательские события и экспонирует данные через реактивные потоки (StateFlow на Android, Published-свойства на iOS).

MVI (Model-View-Intent) — строгое однонаправленное управление состоянием: UI генерирует Intent (намерение), ViewModel редуцирует его в новое State, UI рендерит State. Преимущество MVI — полная предсказуемость: в любой момент можно воспроизвести любой экран, передав соответствующий State. Это существенно упрощает тестирование и отладку сложных сценариев (нарядов-допуска с многошаговым согласованием, например).

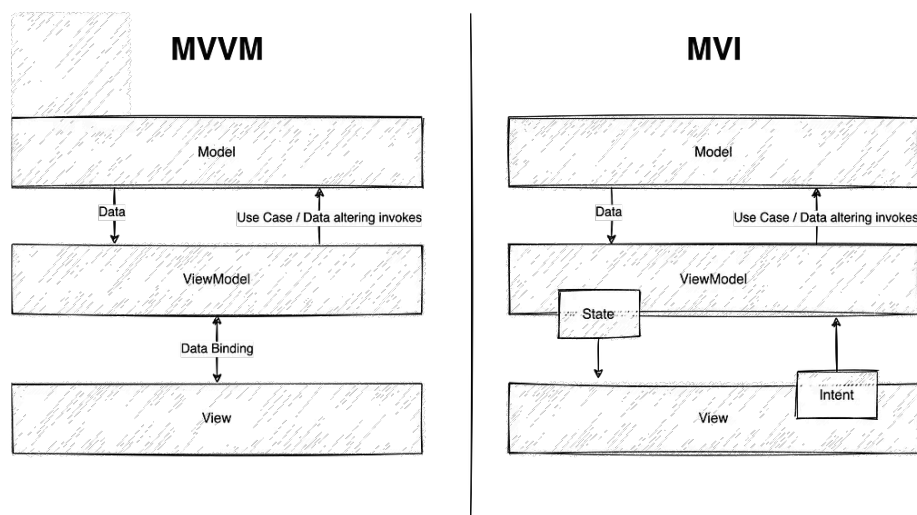


Рисунок 6.2 — Сравнение MVVM и MVI

В КМР-проектах под «Аврору» MVVM/MVI реализуется в общем `commonMain`-модуле: ViewModel на Kotlin Coroutines + StateFlow работает одинаково на Android, iOS и в КМР-модуле, интегрируемом с Qt через Aurora

Multiplatform [34]. Qt-сторона подписывается на состояние через `QFuture` и соответствующие Qt-обёртки, генерируемые плагином `QtBindings` [35].

6.4. Управление корпоративными приложениями: MAM

MAM (Mobile Application Management) — подмножество EMM, управляющее политиками отдельных приложений, а не всем устройством целиком. MAM особенно актуален при BYOD-модели: организация управляет только корпоративными приложениями, не затрагивая личные данные сотрудника.

Типовые MAM-политики для enterprise-приложения в атомной отрасли:

- **Шифрование данных приложения** — независимо от общесистемного шифрования устройства;

- **Запрет copy-paste** из корпоративного приложения в личные (исключает утечку через буфер обмена);

- **Запрет скриншотов** — флаг `FLAG_SECURE` на Android, `allowScreenCapture: false` на iOS блокируют снимок экрана и предпросмотр в переключателе задач;

- **Принудительная версия** — MDM-сервер блокирует доступ приложения к backend при использовании устаревшей версии, принуждая к обновлению;

- **Удалённое уничтожение данных приложения** (selective wipe) — стирает только корпоративные данные без сброса личного устройства к заводским настройкам.

На «Авроре» перечисленные политики реализуются через «Аврора Центр» (EMM) и распространяются на все приложения, установленные через корпоративный канал.

6.5. Безопасность: шифрование, VPN, контейнеризация

6.5.1. Шифрование данных

Enterprise-приложения в атомной отрасли обязаны обеспечивать шифрование чувствительных данных на двух уровнях.

Шифрование в покое (at-rest): локальная база данных шифруется поверх полнодискового шифрования устройства. На Android — `SQLCipher` (AES-256) поверх Room или `EncryptedSharedPreferences` для пар ключ-значение. На «Авроре» — шифрование с ГОСТ-алгоритмами (ГОСТ Р 34.12-2015, «Магма» / «Кузнечик») через интегрированный КриптоПро, что является обязательным требованием для ГИС и КИИ.

Шифрование в движении (in-transit): весь трафик передаётся исключительно по TLS 1.2+. Для взаимодействия со государственными системами обязателен ГОСТ TLS с сертификатами, выданными доверенными российскими УЦ⁴⁰ (Минцифры, «Ростелеком», КристоПро СА). Использование западных корневых центров сертификации для критических систем не допускается.

6.5.2. VPN

Доступ к корпоративным ресурсам из мобильного приложения осуществляется через корпоративный VPN. Для режимных объектов атомной отрасли требуется сертифицированный ФСБ VPN-клиент: ViPNet Client («ИнфоТеКС»), «КристоПро IPsec» или АПКШ «Континент» (поддерживает мобильных клиентов).

На «Авроре» VPN-функциональность интегрирована в «Аврора Центр»: MDM-администратор настраивает VPN-профиль (тип, адрес сервера, сертификаты) и принудительно распространяет его на устройства через EMM-политику.

Per-app VPN — более гранулированная модель: VPN-туннель поднимается только для трафика конкретного приложения, а личный трафик сотрудника (в BYOD/COPE-сценариях) идёт напрямую. Поддерживается на Android Enterprise и iOS, на «Авроре» в контексте СОВО-модели не актуален.

6.5.3. Контейнеризация: Knox и Managed Profile

Samsung Knox — платформа безопасности Samsung, создающая аппаратно изолированный контейнер внутри Android-устройства. Корпоративные приложения и данные в контейнере Knox криптографически изолированы от личного пространства пользователя на уровне ядра. Knox применяется в ряде российских корпораций, использующих устройства Samsung в корпоративном парке.

Android Managed Profile (Work Profile) — стандартный механизм Android Enterprise, создающий изолированный рабочий профиль с отдельным пространством приложений и данных. Управляется MDM без Knox — достаточен для большинства корпоративных BYOD-сценариев.

Для атомной отрасли в СОВО-конфигурации разделение на контейнеры менее актуально: устройство полностью корпоративное, личное использование

⁴⁰УЦ — Удостоверяющий центр. Организация, выпускающая и подтверждающая сертификаты ЭП. В России аккредитованные УЦ перечислены Министерством цифрового развития.

запрещено политикой. Критичнее — аппаратная изоляция, обеспечиваемая Secure Enclave (iOS) или Trusted Execution Environment (Android, «Аврора»).

6.6. Примеры enterprise-приложений в атомной отрасли России

Конкретные мобильные решения, применяемые или разрабатываемые в рамках Госкорпорации «Росатом» и смежных структурах:

Цифровой обход оборудования. Мобильное приложение для обходного персонала АЭС: маршрут обхода с QR/NFC-идентификацией оборудования, фиксация параметров и дефектов, фотоприложения, интеграция с системой ТООР (SAP PM). Сокращает трудозатраты на 30%, объём бумажных актов — на 80% [20].

Электронный наряд-допуск. Мобильное приложение для оформления разрешений на работы повышенной опасности: многошаговое согласование с КЭП ответственных лиц, проверка квалификации персонала через интеграцию с HR-системой, уведомления о статусе наряда. Сокращает время оформления с 30 до 20 минут [20].

Мобильное рабочее место оперативного персонала. Планшет с «Авророй» как замена стационарного АРМ⁴¹: отображение телеметрии с АСУ ТП, журнал событий, переписка в защищённом корпоративном мессенджере, ЭДО. Устройство работает в СОВО-режиме под управлением «Аврора Центр».

Инспекционные приложения. Мобильные решения для инспекторов ядерной и радиационной безопасности: чеклисты проверок, фотофиксация, формирование отчётов в формате, совместимом с регуляторными системами ФСТЭК/Росатома, с КЭП-подписью результатов инспекции.

Все перечисленные решения в защищённом исполнении реализуются на ОС «Аврора» в сочетании с «Аврора Центр», сертифицированным VPN-клиентом и ГОСТ-шифрованием, формируя единый стек, удовлетворяющий требованиям ФСТЭК и ФСБ для объектов КИИ первой категории.

⁴¹ АРМ — автоматизированное рабочее место. Комплекс аппаратного и программного обеспечения, предназначенный для решения задач конкретного специалиста.

Заключение

В ходе написания реферата были рассмотрены ключевые аспекты современных мобильных операционных систем и приложений в контексте их применения в корпоративной и атомной отраслях.

Глобальный рынок мобильных ОС по-прежнему определяется двумя платформами — Android и iOS, каждая из которых выработала собственную модель безопасности и корпоративного управления. Вместе с тем в России сформировался самостоятельный сегмент отечественных решений, обусловленный политикой импортозамещения и санкционным давлением. АОСР-производные системы (KVADRA_OS, РЕД ОС М) закрывают потребительский и часть корпоративного рынка, тогда как Linux-based «Аврора» ОС занимает единственную нишу, где применение зарубежных платформ регуляторно невозможно — объекты КИИ первой категории, государственные информационные системы и структуры, работающие с государственной тайной.

Атомная отрасль оказалась одним из наиболее показательных контекстов для анализа мобильных технологий: жёсткие регуляторные требования ФСТЭК, ФСБ и МАГАТЭ, физическая изоляция технологических сетей и необходимость криптографической защиты данных формируют требования, которым на сегодняшний день соответствует лишь «Аврора» ОС с двойной сертификацией. Практика «Росатома», «Россетей» и других крупных государственных корпораций демонстрирует реальный спрос на мобилизацию рабочих мест — цифровые обходы оборудования, электронные наряды-допуски, интеграцию с АСУ ТП — при условии сохранения полного контроля над программным стеком.

В части мобильных приложений прослеживается устойчивое движение в сторону кросс-платформенных решений. Flutter и React Native зрелые и широко применяемые, однако именно Kotlin Multiplatform демонстрирует наиболее органичный подход для enterprise: разделение бизнес-логики без навязывания единого UI-стека позволяет команде сохранять нативное качество на каждой платформе. Инициатива ОМП по поддержке КМР для «Авроры» через проект Aurora Multiplatform открывает перспективу единой кодовой базы для Android, iOS и «Авроры» одновременно — что для российских корпоративных разработчиков является значимым снижением стоимости поддержки мультиплатформенных приложений.

Ключевой тенденцией ближайших лет остаётся формирование в России концепции «доверенного мобильного рабочего места»: сертифицированное

устройство, отечественная ОС, аттестованный прикладной слой и отечественная криптография в едином верифицированном стеке. Для атомной отрасли эта концепция из желательной постепенно превращается в обязательную с точки зрения регуляторных требований к аттестации информационных систем.

Список использованных источников

1. Number of mobile devices worldwide 2015–2025 [Электронный ресурс]. 2025. URL: <https://www.statista.com/statistics/245501/multiple-mobile-device-ownership-worldwide/> (дата обращения: 30.04.2026).
2. DataReportal. Digital 2025: Global Overview Report [Электронный ресурс]. 2025. URL: <https://datareportal.com/reports/digital-2025-global-overview-report> (дата обращения: 28.04.2026).
3. Google LLC. Android Architecture overview [Электронный ресурс]. 2024. URL: <https://source.android.com/docs/core/architecture> (дата обращения: 01.05.2026).
4. Apple Inc. Apple Platform Security [Электронный ресурс]. 2024. URL: <https://support.apple.com/guide/security/welcome/web> (дата обращения: 01.05.2026).
5. Министерство цифрового развития, связи и массовых коммуникаций РФ. Реестр отечественного программного обеспечения [Электронный ресурс]. 2024. URL: <https://reestr.mindigital.gov.ru/> (дата обращения: 25.04.2026).
6. ООО «Открытая Мобильная Платформа». Аврора — российская мобильная операционная система [Электронный ресурс]. 2024. URL: <https://auroraos.ru/> (дата обращения: 01.05.2026).
7. Google LLC. Flutter: Build apps for any screen [Электронный ресурс]. 2024. URL: <https://flutter.dev/> (дата обращения: 26.04.2026).
8. Meta Platforms, Inc. React Native: Learn once, write anywhere [Электронный ресурс]. 2024. URL: <https://reactnative.dev/> (дата обращения: 26.04.2026).
9. Государственная корпорация «Росатом». Цифровая трансформация Росатома [Электронный ресурс]. 2024. URL: <https://www.rosatom.ru/production/digital/> (дата обращения: 28.04.2026).
10. Mobile Operating System Market Share Worldwide [Электронный ресурс]. 2025. URL: <https://gs.statcounter.com/os-market-share/mobile/worldwide> (дата обращения: 29.04.2026).
11. Google LLC. Android Security Overview [Электронный ресурс]. 2024. URL: <https://source.android.com/docs/security/overview> (дата обращения: 30.04.2026).
12. Google LLC. Android Enterprise [Электронный ресурс]. 2025. URL: <https://developers.google.com/android/work/overview> (дата обращения: 02.05.2026).

13. YADRO / Kvadra. KVADRA_OS — мобильная операционная система [Электронный ресурс]. 2024. URL: <https://yadro.com/kvadraos> (дата обращения: 03.05.2026).
14. ООО «РЕД СОФТ». РЕД ОС М — российская мобильная ОС [Электронный ресурс]. 2024. URL: <https://redosm.red-soft.ru/> (дата обращения: 26.04.2026).
15. ООО «РОСА СОФТ». РОСА Мобайл [Электронный ресурс]. 2026. URL: <https://rosa.ru/mobile/> (дата обращения: 27.04.2026).
16. ООО «Открытая Мобильная Платформа». Аврора ОС — российская мобильная ОС [Электронный ресурс]. 2026. URL: <https://auroraos.ru/> (дата обращения: 30.04.2026).
17. ООО «Открытая Мобильная Платформа». Аврора ОС — российская мобильная ОС. Документация [Электронный ресурс]. 2026. URL: <https://auroraos.ru/documentation> (дата обращения: 28.04.2026).
18. ООО «Открытая Мобильная Платформа». Аврора ОС — российский мобильная ОС. Портал разработчиков [Электронный ресурс]. 2026. URL: <https://developer.auroraos.ru/> (дата обращения: 27.04.2026).
19. International Atomic Energy Agency. Computer Security of Instrumentation and Control Systems at Nuclear Facilities: IAEA Nuclear Security Series No. 33-T. Vienna, 2018.
20. ПАО «Россети Московский регион». Автоматизированная система управления мобильными бригадами [Электронный ресурс]. 2023. URL: <https://rossetimr.ru/client/services/asy/asy-1/> (дата обращения: 29.04.2026).
21. TAdviser. Аврора ОС (ранее SailfishOS). Статья [Электронный ресурс]. 2024. URL: [https://www.tadviser.ru/index.php/%D0%9F%D1%80%D0%BE%D0%B4%D1%83%D0%BA%D1%82:%D0%90%D0%B2%D1%80%D0%BE%D1%80%D0%B0_%D0%9E%D0%A1_\(%D1%80%D0%B0%D0%BD%D0%B5%D0%B5_SailfishOS\)](https://www.tadviser.ru/index.php/%D0%9F%D1%80%D0%BE%D0%B4%D1%83%D0%BA%D1%82:%D0%90%D0%B2%D1%80%D0%BE%D1%80%D0%B0_%D0%9E%D0%A1_(%D1%80%D0%B0%D0%BD%D0%B5%D0%B5_SailfishOS)) (дата обращения: 30.04.2026).
22. JetBrains s.r.o. Kotlin Programming Language [Электронный ресурс]. 2024. URL: <https://kotlinlang.org/> (дата обращения: 29.04.2026).
23. Google LLC. Kotlin and Android — Android Developers [Электронный ресурс]. 2024. URL: <https://developer.android.com/kotlin> (дата обращения: 01.05.2026).
24. Google LLC. Android Studio — The Official IDE for Android [Электронный ресурс]. 2024. URL: <https://developer.android.com/studio> (дата обращения: 02.05.2026).

25. Apple Inc. Swift — A powerful open language that lets everyone build amazing apps [Электронный ресурс]. 2024. URL: <https://swift.org/> (дата обращения: 27.04.2026).
26. Apple Inc. Xcode — Apple Developer [Электронный ресурс]. 2024. URL: <https://developer.apple.com/xcode/> (дата обращения: 28.04.2026).
27. JetBrains s.r.o. Kotlin Multiplatform [Электронный ресурс]. 2024. URL: <https://www.jetbrains.com/kotlin-multiplatform/> (дата обращения: 28.04.2026).
28. JetBrains s.r.o. Compose Multiplatform [Электронный ресурс]. 2024. URL: <https://www.jetbrains.com/lp/compose-multiplatform/> (дата обращения: 03.05.2026).
29. Microsoft Corporation..NET Multi-platform App UI (.NET MAUI) [Электронный ресурс]. 2024. URL: <https://dotnet.microsoft.com/en-us/apps/maui> (дата обращения: 30.04.2026).
30. Google Developers. Progressive Web Apps [Электронный ресурс]. 2024. URL: <https://web.dev/explore/progressive-web-apps> (дата обращения: 25.04.2026).
31. MDN Web Docs / Mozilla. Service Worker API [Электронный ресурс]. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (дата обращения: 03.05.2026).
32. MDN Web Docs / Mozilla. Web app manifests [Электронный ресурс]. 2024. URL: <https://developer.mozilla.org/en-US/docs/Web/Manifest> (дата обращения: 01.05.2026).
33. MDN Web Docs / Mozilla. Push API [Электронный ресурс]. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API (дата обращения: 02.05.2026).
34. ООО «Открытая Мобильная Платформа». Aurora Multiplatform — поддержка КМП для ОС Аврора [Электронный ресурс]. 2024. URL: <https://omprussia.gitlab.io/kmp/docs/book/main.html> (дата обращения: 30.04.2026).
35. ООО «Открытая Мобильная Платформа». QtBindings — интеграция Kotlin Multiplatform с Qt/QML для ОС Аврора [Электронный ресурс]. 2024. URL: <https://omprussia.gitlab.io/kmp/docs/book/plugins/qt-bindings/main.html> (дата обращения: 02.05.2026).
36. ООО «Открытая Мобильная Платформа». Aurora KInterop — взаимодействие КМП с системой ОС Аврора [Электронный ресурс]. 2024. URL:

<https://omprussia.gitlab.io/kmp/docs/book/libs/aurora-kinterop/main.html> (дата обращения: 29.04.2026).

37. ООО «Открытая Мобильная Платформа». Compose Multiplatform для ОС Аврора [Электронный ресурс]. 2024. URL: <https://omprussia.gitlab.io/kmp/docs/book/plugins/cmp/main.html> (дата обращения: 26.04.2026).
38. ООО «Открытая Мобильная Платформа». BuildTools — инструменты сборки КМР/СМР для ОС Аврора [Электронный ресурс]. 2024. URL: <https://omprussia.gitlab.io/kmp/docs/book/plugins/build-tools/main.html> (дата обращения: 25.04.2026).
39. Fastlane Tools. fastlane — Automate your mobile app deployments [Электронный ресурс]. 2024. URL: <https://fastlane.tools/> (дата обращения: 25.04.2026).
40. Мартин Р.С. Чистая архитектура. Искусство разработки программного обеспечения. Санкт-Петербург: Питер, 2018. 352 с.

Приложение А

Доклад по теме

1. Цели и задачи работы

Цель настоящей работы — комплексное исследование мобильных операционных систем и мобильных приложений с позиции их применения в корпоративной среде, прежде всего в организациях атомной отрасли России.

Для достижения цели поставлены следующие **задачи**:

- рассмотреть архитектуру и характеристики ведущих мобильных операционных систем — Android, iOS, а также российских разработок;
- проанализировать особенности применения мобильных ОС в корпоративной среде: модели развёртывания, управление устройствами, требования к безопасности;
- изучить специфику внедрения мобильных технологий в атомной отрасли: цифровые обходы, интеграция с АСУ ТП, требования регуляторов;
- классифицировать виды мобильных приложений и сравнить подходы к их разработке;
- выявить особенности разработки и эксплуатации корпоративных приложений с акцентом на отечественные инструменты.

Актуальность темы обусловлена ускоренной цифровизацией производственных процессов в условиях импортозамещения и необходимостью обеспечения информационной безопасности критической информационной инфраструктуры (КИИ)⁴².

2. Рынок мобильных ОС

2.1. Глобальный рынок

По итогам 2024 года мировой рынок мобильных операционных систем характеризуется явной двухполюсной структурой. Android занимает около 72–78 % рынка смартфонов, iOS — около 27–22 % в зависимости от региона. Прочие платформы суммарно не превышают 1 %.

⁴²КИИ — критическая информационная инфраструктура; понятие закреплено Федеральным законом № 187-ФЗ от 26.07.2017.

Таблица 2.1 — Распределение глобального рынка мобильных ОС (2024 г.)

Операционная система	Доля рынка (глобально)	Основной производитель
Android	72–78 %	Google / AOSP-сообщество
iOS	22–27 %	Apple Inc.
Прочие	< 1 %	KaiOS, HarmonyOS и др.

В корпоративном сегменте соотношение смещается в сторону iOS в странах Западной Европы и Северной Америки, тогда как в России корпоративный рынок исторически ориентирован на Android-устройства и сейчас целенаправленно переходит на отечественные платформы.

3. Android: архитектура и особенности

Android построен на многоуровневой архитектуре поверх ядра Linux. Уровни, от нижнего к верхнему:

1. **Ядро Linux** (Linux Kernel) — управление аппаратными ресурсами, драйверами, безопасностью на уровне процессов.
2. **Аппаратный уровень абстракции** (HAL, Hardware Abstraction Layer) — унифицированный интерфейс для взаимодействия системы с конкретным железом.
3. **Android Runtime** (ART) — виртуальная машина с AOT-компиляцией байт-кода Dalvik; заменила Dalvik VM начиная с Android 5.0.
4. **Нативные библиотеки C/C++** (libc, OpenGL ES, SQLite, WebKit и др.).
5. **Java API Framework** — набор API, предоставляемых приложениям: менеджеры активностей, пакетов, уведомлений, телефонии и т.д.
6. **Системные приложения** — стандартный набор (звонки, SMS, браузер, настройки).

Ключевая особенность Android — открытость исходного кода (AOSP, Android Open Source Project), что позволяет создавать производные платформы без лицензионных отчислений Google. Именно на AOSP базируются российские ОС KVADRA_OS и РЕД ОС М.

Существенный недостаток Android-экосистемы — **фрагментация**: по данным на 2024 год, одновременно в эксплуатации находятся устройства под управлением версий от Android 10 до Android 14, что усложняет поддержку корпоративных приложений и своевременное получение патчей безопасности.

4. iOS: архитектура и особенности

iOS (с 2019 г. iPadOS для планшетов) построена на основе Darwin — Unix-подобного ядра XNU (гибрид Mach и BSD). Архитектура iOS также многоуровневая:

1. **Core OS** — ядро Darwin, криптографические примитивы, Bluetooth, Wi-Fi.
2. **Core Services** — фундаментальные сервисы: iCloud, Core Data, Core Location, Core Motion.
3. **Media** — AVFoundation, Core Image, Core Audio, Metal (GPU API).
4. **Cocoa Touch** — UIKit, MapKit, GameKit; уровень, с которым непосредственно взаимодействуют приложения.

В отличие от Android, iOS — **закрыва́тая** система: установка приложений возможна только через App Store (или корпоративный MDM-профиль). Это повышает безопасность, но ограничивает гибкость. Каждое приложение выполняется в изолированной **sandbox**-среде с жёсткими ограничениями на доступ к системным ресурсам. Обновления ОС выпускаются централизованно Apple и распространяются синхронно на все поддерживаемые устройства, что практически исключает проблему фрагментации.

5. Российские мобильные операционные системы

На отечественном рынке выделяются две группы мобильных ОС: производные AOSP и дистрибутивы на базе Linux.

Таблица 5.1 — Российские мобильные операционные системы

ОС	Основа	Разработчик	Целевой сегмент
KVADRA_OS	AOSP (Android)	НТЦ ИТ РОСА / КНТТ	Планшеты, корп. устройства
РЕД ОС М	AOSP (Android)	РЕД СОФТ	Корпоративный, госсектор
Аврора ОС	Sailfish OS / Linux	Открытая мобильная платформа	Гос. и корп. сектор, КИИ
ROSA Mobile	Linux (Mer/Nemo)	РОСА	Исследовательский

KVADRA_OS и **РЕД ОС М** сохраняют совместимость с Android-приложениями (APK), что облегчает миграцию. Однако Google Mobile Services (GMS) в них недоступны, что требует переработки приложений, зависящих от сервисов Google.

Аврора ОС принципиально отличается: её ядро построено на Linux без использования AOSP, а API несовместим с Android. Платформа развивается «Открытой мобильной платформой» (ОМП) при поддержке «Ростелекома» и ориентирована на применение в государственных структурах и на объектах КИИ, где использование иностранных платформ ограничено.

6. Корпоративная среда: модели развёртывания

В корпоративной мобильности применяются три основные модели управления устройствами, отличающиеся балансом между гибкостью для сотрудника и контролем со стороны организации.

Таблица 6.1 — Модели развёртывания корпоративных мобильных устройств

Модель	Расшифровка	Владелец устройства	Применение
BYOD	Bring Your Own Device	Сотрудник	Офисный персонал, частичная занятость
COPE	Corporate-Owned, Personally Enabled	Организация	Линейный персонал с личными нуждами
COBO	Corporate-Owned, Business Only	Организация	КИИ, производство, режимные объекты

Для управления корпоративными устройствами применяются системы **MDM** (Mobile Device Management) и более широкие **EMM** (Enterprise Mobility Management), включающие **MAM** (управление приложениями) и **MIM** (управление информацией). На объектах КИИ, в том числе в атомной отрасли, доминирует модель COBO, поскольку она обеспечивает полный контроль над программной средой устройства.

7. Аврора ОС в корпоративной среде

Аврора ОС обладает архитектурными особенностями, выгодно отличающими её от Android в контексте КИИ. Ключевые компоненты:

- **Ядро Linux** — стабильная и аудируемая основа без фирменных слоёв AOSP.
- **Wayland Compositor** (Lipstick) — замена X11/SurfaceFlinger; управляет отрисовкой пользовательского интерфейса.

- **Sailfish Silica / Qt** — UI-фреймворк на базе QML и Qt; собственный, не связанный с Android API.
- **D-Bus** — межпроцессное взаимодействие системных демонов.
- **MDM-агент** — встроенный агент управления, позволяющий централизованно конфигурировать, блокировать и обновлять устройства.

Средства управления Аврора ОС объединены в **Аврора Центр** — корпоративную консоль MDM. Она обеспечивает инвентаризацию устройств, дистанционную установку приложений, политики паролей и шифрования, а также удалённое уничтожение данных (remote wipe).

Важным преимуществом является сертификация ФСТЭК⁴³ России, что позволяет использовать Аврора ОС для обработки сведений ограниченного доступа.

8. Мобильные технологии в атомной отрасли

8.1. Цифровые обходы оборудования

Одним из наиболее востребованных приложений мобильных технологий на предприятиях Росатома являются **системы цифровых обходов** (инспекционные маршруты). Традиционно обходы выполнялись с бумажными чек-листами, что порождало задержки в передаче данных и риск ошибок при ручном вводе.

Цифровой обход на планшете с Аврора ОС позволяет:

- сканировать QR-коды или NFC-метки на единицах оборудования для автоматической идентификации;
- вносить показания приборов и выявленные замечания непосредственно в мобильное приложение;
- прикреплять фотографии дефектов;
- синхронизировать данные с корпоративными системами (EAM, ERP) после выхода из зоны с ограниченным радиодоступом.

8.2. Интеграция с АСУ ТП

Интеграция мобильных решений с АСУ ТП⁴⁴ реализуется через специализированные шлюзы и OPC UA-серверы, изолирующие технологический уровень от корпоративного. Мобильные устройства никогда не подключаются напрямую к SCADA-системам; взаимодействие осуществляется по схеме:

⁴³ ФСТЭК — Федеральная служба по техническому и экспортному контролю; орган, уполномоченный выдавать сертификаты соответствия требованиям защиты информации.

⁴⁴ АСУ ТП — автоматизированная система управления технологическим процессом.

Передаваемые данные — как правило, телеметрия для отображения (только чтение); команды управления с мобильных устройств исключены из архитектуры безопасности.

9. Требования безопасности КИИ и сетевая сегментация

Объекты атомной отрасли относятся к объектам КИИ первой категории значимости. Ключевые регуляторные требования:

Таблица 9.1 — Регуляторные требования к мобильным решениям на объектах КИИ

Документ / регулятор	Основное требование для мобильных устройств
ФЗ № 187-ФЗ (КИИ)	Использование ПО и оборудования из реестра российской продукции
Приказ ФСТЭК № 239	Сегментация сети, контроль съёмных носителей, мониторинг инцидентов
НП-026-16 (Росатом)	Запрет несанкционированных подключений к АСУ ТП
Приказ ФСБ № 378	Шифрование каналов связи сертифицированными СКЗИ

Сетевая сегментация (Network Segmentation) является фундаментальным принципом защиты. Сеть предприятия делится как минимум на три уровня:

1. **Корпоративная сеть** (уровень L3) — доступ с мобильных устройств через Wi-Fi или мобильный Интернет с VPN; применяется MDM-контроль.
2. **DMZ** (демилитаризованная зона) — граничный сегмент; здесь размещаются шлюзы, обеспечивающие однонаправленную передачу данных из промышленной сети.
3. **Промышленная (технологическая) сеть** (уровень L1/L2) — физически изолирована; мобильные устройства в неё не включаются.

Для мобильных устройств на объектах Росатома обязательно применение:

- сертифицированных СКЗИ⁴⁵ (например, ViPNet) для VPN;

⁴⁵СКЗИ — средства криптографической защиты информации.

- MDM-политик, запрещающих установку стороннего ПО, использование Bluetooth и личных облачных сервисов;
- Аврора ОС как сертифицированной платформы вместо Android или iOS.

10. Виды мобильных приложений

Мобильные приложения классифицируются по архитектурному подходу к разработке. Каждый подход предполагает различный баланс между производительностью, стоимостью разработки и охватом платформ.

Таблица 10.1 — Сравнение видов мобильных приложений

Вид	Исполнение	Доступ к API ОС	Доставка
Нативное	Машинный код / ART	Полный	App Store / MDM
Кросс-платформенное	Нативное (компиляция) или JIT (интерпретация)	Полный / частичный	App Store / MDM
Гибридное	WebView	Через JS-мост	App Store / MDM
PWA	Браузер	Ограниченный	URL (без магазина)

11. Нативная разработка

Нативные приложения создаются с использованием официальных SDK конкретной платформы и компилируются в машинный код (или байт-код, исполняемый нативной средой платформы).

Таблица 11.1 — Языки и инструментарий нативной разработки

Платформа	Язык	Инструментарий
Android	Kotlin (основной), Java	Android Studio, Gradle
iOS	Swift (основной), Obj-C	Xcode, SwiftUI / UIKit
Аврора ОС	C++ / Qt, Python	Qt Creator, Аврора SDK

Нативный подход обеспечивает максимальную производительность и полный доступ ко всем возможностям платформы: камере, NFC, Bluetooth, датчикам. Это критически важно для промышленных приложений, работаю-

щих с периферийным оборудованием. Основной недостаток — необходимость поддерживать отдельную кодовую базу для каждой платформы.

Для Аврора ОС использование Qt/C++ дополнительно выгодно тем, что Qt имеет давнюю историю применения во встраиваемых и промышленных системах, что упрощает интеграцию с существующим ПО предприятий.

12. Кросс-платформенная разработка

Кросс-платформенные фреймворки позволяют использовать единую (или почти единую) кодовую базу для нескольких платформ.

Таблица 12.1 — Сравнение кросс-платформенных фреймворков

Фреймворк	Язык	Рендеринг UI	Поддержка Авроры
Flutter	Dart	Собственный движок (Skia/Impeller)	Экспериментальная
React Native	JavaScript / TypeScript	Нативные компоненты ОС	Отсутствует
KMP / CMP	Kotlin	Нативный (expect/actual) или Compose	Да (через ОМП)

Flutter компилирует Dart-код в нативный ARM-код и рисует UI через собственный графический движок, не используя виджеты платформы. Это обеспечивает визуальную согласованность между платформами, но усложняет интеграцию с системным UI.

React Native использует JavaScript-мост для вызова нативных компонентов, что создаёт накладные расходы на коммуникацию между JS-поток и нативным потоком.

Kotlin Multiplatform (KMP) — принципиально иной подход: только **бизнес-логика** (сеть, база данных, доменный слой) является общей, тогда как UI разрабатывается нативно для каждой платформы. **Compose Multiplatform (CMP)** расширяет KMP, добавляя возможность разделять и UI-код на базе Jetpack Compose.

13. Аврора ОС и Kotlin Multiplatform

Аврора Мультиплатформа — инициатива «Открытой мобильной платформы» по интеграции KMP с Аврора ОС. Цель — позволить командам

разрабатывать мобильные приложения для Android, iOS и Аврора ОС с максимально общей кодовой базой.

Таблица 13.1 — Распределение кода в проекте КМР с поддержкой Аврора ОС

Слой приложения	Реализация в КМР + Аврора
Сетевой (HTTP, REST)	Общий Kotlin-код (Ktor)
Локальная БД	Общий Kotlin-код (SQLDelight)
Бизнес-логика / ViewModel	Общий Kotlin-код
UI — Android	Jetpack Compose (Kotlin)
UI — iOS	SwiftUI / UIKit (Swift)
UI — Аврора ОС	Qt/QML (C++) через КМР-биндинги

Архитектурно биндинг работает следующим образом: КМР-библиотека компилируется в нативный бинарный файл (.so) для платформы Аврора (aarch64-linux); Qt-приложение подключает эту библиотеку и вызывает её функции через С-интерфейс. Таким образом, бизнес-логика пишется один раз на Kotlin, а UI остаётся нативным для каждой ОС.

Это решение является стратегически важным для атомной отрасли: оно позволяет параллельно поддерживать приложения для Android-устройств (общего назначения, административный персонал) и для Аврора ОС (режимные объекты, операционный персонал), не дублируя значительную часть кода.

14. Заключение

В ходе работы достигнута поставленная цель и решены все задачи.

- Архитектура мобильных ОС.** Рассмотрены многоуровневые архитектуры Android и iOS, проанализированы отечественные платформы: KVADRA_OS и РЕД ОС М (производные AOSP) и Аврора ОС (Linux-платформа). Показано, что для объектов КИИ Аврора ОС является единственной сертифицированной отечественной платформой.
- Корпоративная среда.** Сравнение моделей BYOD / COPE / COBO показало, что для промышленных предприятий и режимных объектов обоснована модель COBO с применением MDM. Аврора ОС обеспечивает встроенные инструменты управления через Аврора Центр.
- Атомная отрасль.** Определены основные сценарии применения: цифровые обходы оборудования, сбор телеметрии, работа с технической документацией. Установлено, что интеграция с АСУ ТП допу-

стима только через однонаправленные шлюзы в DMZ. Требования ФСТЭК и Росатома фактически предписывают использование Авроры ОС.

4. **Классификация приложений.** Систематизированы нативный, кросс-платформенный, гибридный подходы и PWA; для промышленных приложений приоритетен нативный подход.
5. **Корпоративные приложения и Аврора Мультиплатформа.** КМР позволяет разделять бизнес-логику между Android и Аврора ОС, снижая затраты на разработку при сохранении нативного UI для каждой платформы. Это стратегически перспективное направление для цифровизации Росатома в условиях импортозамещения.

Практическая значимость результатов состоит в возможности их применения при обосновании выбора мобильной платформы и стека разработки для корпоративных проектов предприятий с высокими требованиями к информационной безопасности.

Приложение Б

Графический материал

Доклад на тему:

«Современные мобильные операционные системы, современные приложения и перспективы развития приложений для мобильных устройств разработки ИТ предприятий в Атомной Отрасли»

Студент группы ИКБО-50-25 **Багинян А.В.**

Руководитель **Рогоза В.М.**

Цели и задачи работы

Цель работы: систематизированный анализ современного состояния мобильных операционных систем и приложений с акцентом на их применение в корпоративной и атомной отраслях

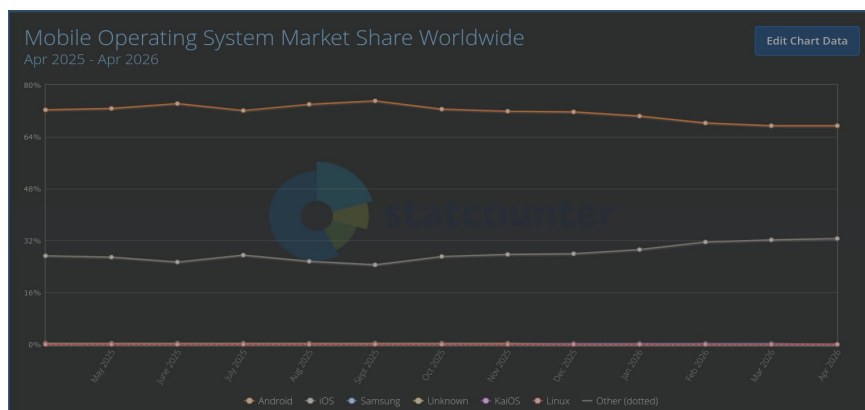
Задачи работы:

- 1) рассмотреть архитектуру и характеристики мобильных ОС;
- 2) проанализировать особенности применения мобильных ОС в корпоративной среде;
- 3) изучить специфику внедрения мобильных технологий в атомной отрасли;
- 4) классифицировать виды мобильных приложений;
- 5) выявить особенности разработки и эксплуатации корпоративных (enterprise) приложений

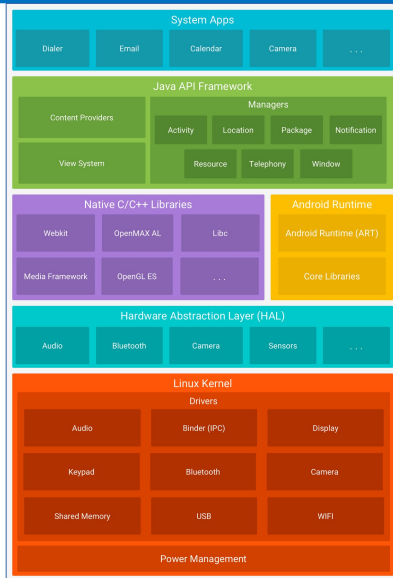
2

Рынок ОС. Глобальный рынок

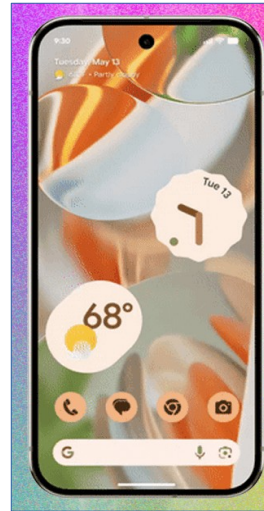
72% Android · 28% iOS.



3

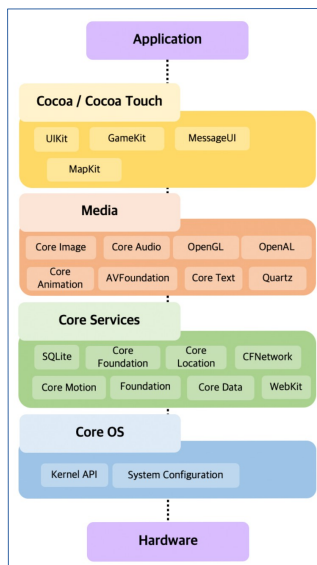


Архитектура Android



Android 16

4



Архитектура iOS



iOS

5

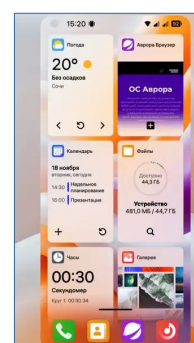
	База	Сертификат
KVADRA_OS	AOSP	Реестр МЦ
РЕД ОС М	AOSP	ФСТЭК УД4
Аврора ОС	Linux	ФСТЭК + ФСБ



KvadraOS



РЕД ОС М



Аврора ОС

6

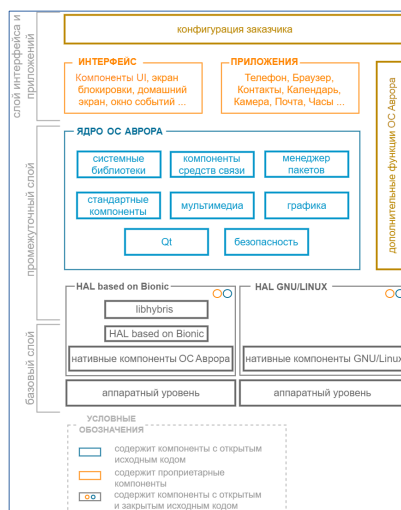
BYOD — Bring Your Own Device

COPE — Corporate-Owned, Personally Enabled

COBO — Corporate-Owned, Business Only

Параметр	BYOD	COPE	COBO
Расшифровка	Bring Your Own Device	Corporate-Owned, Personally Enabled	Corporate-Owned, Business Only
Владелец устройства	Сотрудник	Организация	Организация
Личное использование	Да	Да (ограниченно)	Нет
Контроль ИТ-отдела	Частичный	Полный	Полный
Типичный сценарий	Офисные сотрудники	Менеджеры среднего звена	Промышленность, госсектор
Стоимость для компании	Низкая	Средняя	Высокая

7



- Единственная ОС с ФСТЭК+ФСБ

Архитектура Аврора ОС

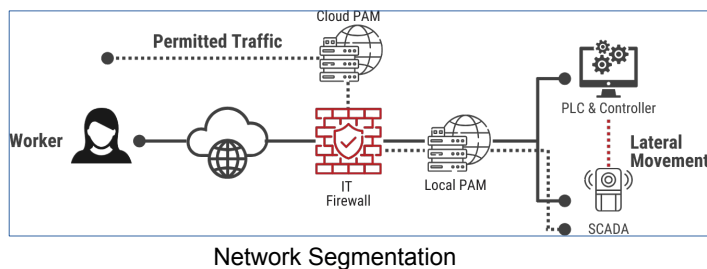
8

- Цифровые обходы оборудования
- Электронные наряды-допуски
- Интеграция с АСУ ТП



9

- физическая изоляция технологической сети от корпоративной и внешнего интернета (air gap)



10

Приложения. Виды приложений

Нативные — Скорость
Кросс-платформа — Экономия
PWA — Доступность



11

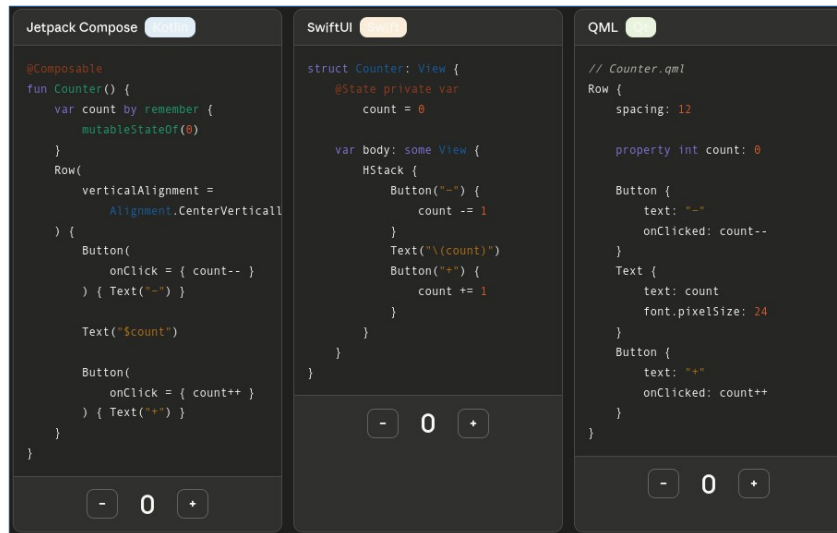
Приложения. Нативные решения

Таблица 11.1 — Языки и инструментарий нативной разработки

Платформа	Язык	Инструментарий
Android	Kotlin (основной), Java	Android Studio, Gradle
iOS	Swift (основной), Obj-C	Xcode, SwiftUI / UIKit
Аврора ОС	C++ / Qt, Python	Qt Creator, Аврора SDK

Основные фреймворки

12



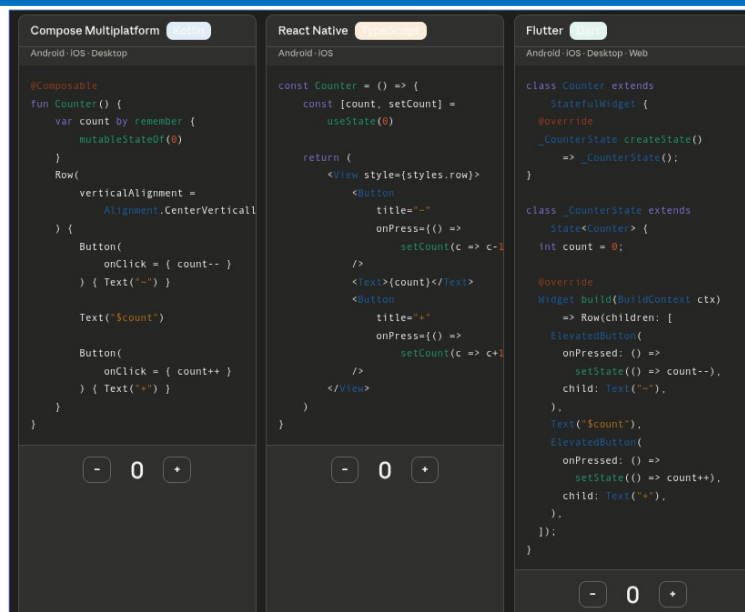
13

Таблица 12.1 — Сравнение кросс-платформенных фреймворков

Фреймворк	Язык	Рендеринг UI	Поддержка Авроры
Flutter	Dart	Собственный движок (Skia/Impeller)	Экспериментальная
React Native	JavaScript / TypeScript	Нативные компоненты ОС	Отсутствует
KMP / CMP	Kotlin	Нативный (expect/actual) или Compose	Да (через QtBindings)

Основные фреймворки

14

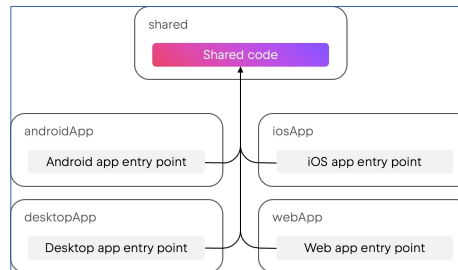


15

```

1 project/
2   └─ shared/           # Общий KMP-модуль
3       └─ commonMain/   # Бизнес-логика (Domain + Data)
4           └─ androidMain/ # Android-специфика
5               └─ iosMain/  # iOS-специфика
6                   └─ linuxArm64Main/ # Аврора-специфика (KInterop)
7   └─ androidApp/        # Android UI (Jetpack Compose)
8       └─ iosApp/        # iOS UI (SwiftUI)
9           └─ auroraApp/  # Аврора UI (Qt/QML)
10               └─ src/
11                   └─ main.qml
12                   └─ KmpBridge.cpp # Qt-обёртки, генерируемые
QtBindings
  
```

Типовой KMP проект с Авророй



Структура KMP проекта

16

Заключение

Выполнены поставленные задачи:

- 1) рассмотрена архитектура мобильных ОС;
- 2) проанализированы особенности применения мобильных ОС в корпоративной среде;
- 3) изучена специфика внедрения мобильных технологий в атомной отрасли;
- 4) классифицированы виды мобильных приложений;
- 5) выявлены особенности разработки и эксплуатации корпоративных (enterprise) приложений

17

Благодарю за внимание

18